NOTTINGHAM TRENT UNIVERSITY
SCHOOL OF SCIENCE AND TECHNOLOGY

Requirements Elicitation and Prototype Evaluation:

Timetabling Requests Management

by

Mohid Nadeem | N1433045

in

2026

Project report in part fulfilment
of the requirements for the degree of
Master of Science
In
Computer Science

I hereby declare that I am the sole author of this report. I authorize the Nottingham Trent University to lend this report to other institutions or individuals for the purpose of scholarly research.

I also authorize the Nottingham Trent University to reproduce this report by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

Signature

Mohid Nadeem

ABSTRACT

Timetable management within university environments frequently relies on fragmented communication channels such as email, spreadsheets, and separate viewing systems, which can result in delayed updates, missed notifications, and unresolved scheduling conflicts. While existing systems at Nottingham Trent University (NTU) allow students and staff to view generated timetables, limited facility exists for structured communication surrounding constraint submission, change requests, and update notifications between stakeholders. This project addresses that gap by investigating, designing, and evaluating a communication-focused timetabling support prototype intended to improve coordination between lecturers, and administrative staff, and students.

A hybrid requirement elicitation approach, combining interviews, questionnaires, and observation, was used to gather functional and non-functional requirements from student and academic staff stakeholders. These findings helped with the design of system workflows, use cases, and interface wireframes, which were then developed into a working prototype using an iterative-incremental development approach. The prototype was implemented in functional increments covering constraint submission, timetable change request handling, and centralised communication notifications, with each increment tested and refined based on ongoing feedback.

The developed prototype was evaluated through a combination of incremental, conversation-based evaluation sessions with a lecturer stakeholder, a System Usability Scale based questionnaire and feature-specific satisfaction ratings completed by Timetabling Team members following a live demonstration, and a targeted usability survey of the automated email notifications completed by students.

Findings indicated that the prototype was positively received across all three stakeholder groups: Timetabling Team respondents rated the system as easy to use and low in complexity or inconsistency; students rated the clarity and usefulness of notification emails highly; and iterative feedback from the lecturer stakeholder led directly to a significant structural refinement of the prototype through development. These results suggest that a communication-focused approach to timetable support, developed through active, continuous stakeholder involvement rather than a single upfront requirements phase, offers a practical and well-received foundation for future improvements to NTU's existing timetabling infrastructure.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project supervisor, **Neil Sculthrope**, for his continuous guidance, feedback, and support throughout the duration of this project. His advice during our regular meetings helped me stay focused on the project objectives and improved the overall direction of both the requirement elicitation and prototype development stages.

I would also like to thank the students, lecturers, and administrative staff who took part in the interviews, questionnaires, and usability testing sessions. Their willingness to share their time, experiences, and honest feedback was essential in shaping the requirements and evaluating the prototype developed as part of this project.

Finally, I would like to thank my family and close friends for their encouragement and support throughout my studies at Nottingham Trent University.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1. Background

Timetabling system is a critical part of educational institutes, as it allows organizing classes, allocating rooms, managing schedules and ensuring coordination between students, lecturers, and admin staff. Despite the availability of digital system at NTU, communication surrounding timetable managements remains a significant challenge. The issues such as delayed updates, timetable clashes, lack of centralized communication and change requests can affect staff and students in a way that can result in confusion, missed classes or scheduling conflicts.

Many institutions still rely on emails, spreadsheets, and messaging platforms for communication and coordination between departments. While these methods may work for small-scale scenarios like for Academy centers, they become difficult to manage as the number of users, modules, and scheduling constraints increase. For example, lecturers may require preferred teaching slots, while admins must ensure efficient room / resource allocation, and students may prioritize avoiding timetable clashes (especially when they do have to opt for optional modules.)

As for the scope of this project, keeping Nottingham Trent University (NTU) a focus, where NTU does have an application where students and staff can easily access the timetable, the proposed project aims to address the coordination challenges through a prototype development of a communication-focused timetable support system. This project focuses on improving the interaction processes regarding timetable management and is intended to provide a prototype where stakeholders can submit scheduling constraints, can request timetable changes, and receive updates.

The project not only focuses on developing a prototype but also on the Requirement Elicitation and Prototype Evaluation. Requirement Elicitation techniques such as Interviews, and Questionnaires will be used to better understand about the needs of the stakeholders, while Usability Testing and the System Usability Scale will be used to evaluate the effectiveness of the prototype.

1.2. Problem Statement

The core problem this project addresses is not the generation of timetables, but the coordination and communication processes that surround them. As the students, lecturers, and administrative staff each have different priorities, and where a lack of centralised communication can lead to missed updates, timetable clashes, and general confusion.

Without a dedicated system for handling these interactions, stakeholders are left relying on fragmented and inconsistent communication channels like Emails, Microsoft Forms, or Sheets. This project therefore investigates whether a purpose-built, communication-focused prototype can improve how scheduling constraints, change requests, and notifications are handled within an NTU-style timetabling environment.

1.3. Aims and Objectives

Aim

To design, develop, and evaluate a communication-focused timetabling support prototype for NTU that improves stakeholder interaction, scheduling coordination, and timetable change management within a university environment.

Objectives

The project objectives are defined as follows:

1. **Conduct a literature review** of requirement elicitation and prototype evaluation techniques relevant to communication-focused system design.

2. **Gather and analyse stakeholder requirements** from students and academic staff using interviews, questionnaires, and observation, producing a documented set of functional and non-functional requirements.

3. **Design a prototype architecture and user workflow**, including use cases and interface wireframes, based on the requirements gathered.

4. **Develop the prototype**, implementing constraint submission, timetable change request, and notification features as separate, testable increments.

5. **Evaluate the prototype** using usability testing sessions and the System Usability Scale (SUS), collecting measurable feedback from stakeholder participants.

6. **Analyse the evaluation results and refine the prototype,** accordingly, producing a final version of the system ready for submission.

Each objective is measurable through a defined deliverable (as outlined in the Project Plan's task table) and is scheduled against a timeframe, ensuring the project remains achievable within the constraints of an academic dissertation.

To achieve this, the project will follow an iterative-incremental development approach, allowing continuous refinement of the system based on stakeholder feedback and usability evaluation.

1.4. Scope and Context

This project is scoped around the communication aspects of timetabling rather than the generation of timetables themselves. Timetable optimisation and scheduling algorithms, fall outside the scope of this project. Instead, the project focuses on the interaction layer: how stakeholders raise issues, request changes, and stay informed.

Therefore, the project focuses on highlighting current issues / challenges faced by stakeholders, coming up with a solution closer to stakeholder needs, and developing a prototype that can serve as foundation for future improvements to the NTU timetable system.

The project is about setting up a development approach, go through software development lifecycle: elicitation of requirements, system design and UI decisions, development of a prototype and evaluation towards the end followed by improvement in the prototype.

1.5. Report Structure

The remainder of this report is structured as follows.

- Chapter 2 reviews existing literature relating to timetabling systems, requirement elicitation, prototype evaluation, and software development approaches.

- Chapter 3 outlines the requirement elicitation process carried out for this project. The requirements are gathered through Interviews taken from lecturer, and through surveys from timetabling team and the students.

- Chapter 4 outlines the requirement elicitation process carried out for this project. The requirements are gathered through Interviews taken from lecturer, and through surveys from timetabling team and the students. Then moves to design and implementation details.

- Chapter 5 presents the results of the usability evaluation, through meetings with lecturer, SUS evaluation with timetabling team, and surveys from students.

- And the report concludes with the Chapter 6 having a discussion of findings, limitations, and recommendations for future work.

CHAPTER 2: LITERATURE REVIEW

2.1. Introduction

2.1.1. Purpose

The main purpose of this Literature Review is to review existing requirement elicitation and prototype evaluation techniques, and to decide which development approach best suits the proposed project "Timetabling Requests Management". The review also aims to identify the communication and usability challenges that stakeholders commonly face within academic scheduling environments.

By reviewing and analyzing previous studies, this review helps establish a decision regarding the techniques and approach to be followed throughout the project phase. Furthermore, this review is intended to justify the decisions made.

2.1.2. Scope

The literature review focuses on the project regarding university timetabling system (specifically NTU, Nottingham Trent University) and the communication challenges associated with the timetable management processes. The review covers research related to stakeholder interaction, requirement elicitation, usability testing, and software development approaches commonly applied.

The scope includes both qualitative and quantitative techniques used in requirements engineering, such as interviews, questionnaires and observation methods. It then covers the prototype evaluation techniques focusing on usability testing and the System Usability Score (SUS), alongside justifying the selection of an iterative-incremental approach for the development process.

2.1.3. Focus Area

The review does not focus deeply on timetable optimization algorithms or any improvement in timetable generation systems, as the primary focus of this project is communication efficiency and stakeholder interaction rather than an improved or automated scheduling generation.

So, the focus area of this literature review is the investigation of methods that support effective communication and user-centred development for a university timetabling system. This review focuses on three key areas: requirement elicitation techniques, prototype evaluation methods, and development approaches. Requirement elicitation is explored to identify effective ways of understanding stakeholder needs and communication challenges they face. Prototype evaluation methods are reviewed to understand how usability and user experience can be measured and improved through testing and feedback. Moreover, development approaches are reviewed to determine suitable model for developing a continuously refined system.

2.2. Timetabling Systems and Communication Challenges

2.2.1. Traditional Timetabling Systems

Timetabling systems are an important component of educational institutions, as they help manage class scheduling, room allocation, lecturer availability, and coordination between multiple academic activities. Traditionally, timetable management was performed manually using spreadsheets, printed schedules, or isolated database systems. Although these methods may function perfect within smaller environments, such as a university with one campus or limited departments. But these methods become difficult to maintain as institutional size and scheduling complexity grow.

Most of the research on university timetabling has focused on optimization and scheduling efficiency which is undoubtedly an important task. Such as, Burke et al. (1997), in their review of university timetabling problems, discussed how many existing systems emphasize constraint satisfaction and timetable generation algorithms. His work included heuristic and mathematical models to resolve the problems in concern. Similarly, McCollum (2007) highlighted that timetabling problems are often treated as technical optimization challenges involving room allocation, scheduling conflicts, and resource management. It is important to create solutions to improve computational efficiency, but the communication related issues experienced by stakeholders during timetable management are mostly overlooked.

Also, my observation as a student suggests that many institutions rely on fragmented communication approaches for timetable updates and modifications. Information is mostly shared through emails, notice boards, spreadsheets, or separate online systems, which can create delays and inconsistencies. As with passing time, majority prefer digital systems for most of the tasks, and universities have also become increasingly digitalized, users expect timetable systems to support not only scheduling functionality but also centralized communication and interaction.

Gould and Lewis (1985) argued that systems designed without continuous user involvement frequently fail to meet practical user expectations. This is because of the changing trends and preferences. One more reason can be, users from different environments may have different expectations. If only a particular group or only developers themselves get the requirements listed for a product, they may lose the viewpoints of different stakeholders, that will result in dissatisfaction for many users later. This is also relevant within academic scheduling environments where multiple stakeholder groups interact with timetable information differently.

2.2.2. Communication Challenges

Communication can be a challenge within timetable management processes. Universities operate in dynamic environments where timetable changes, or there can be lecturer availability issues, that may result in room reallocations, and scheduling conflicts frequently occur. When communication regarding these changes is ineffective, stakeholders may receive outdated information, miss important updates, or face timetable clashes that would negatively impact academic activities.

Communication problems within timetabling systems extend beyond technical scheduling issues and are connected to stakeholder interaction, requirement gathering, and usability concerns.

This is why the proposed project "Timetabling Requests Management" focuses on addressing these communication related challenges through the development of a prototype that focuses on the coordination amongst users. NTU has a system that helps staff for generating timetable, students and teachers to view their respective lectures and rooms allocated. The proposed prototype system aims to improve the interaction between students, lecturers, and administrative staff during timetable management processes. The project focuses on improving requirement collection, timetable change request handling, and update notifications. By adopting suitable requirement elicitation and usability evaluation techniques during development, the project aims to produce a prototype that aligns more closely with stakeholder needs while improving overall communication efficiency within academic scheduling environment.

2.2.3. Stakeholder Involvement in Academic Scheduling

Nuseibeh and Easterbrook (2000) put across a point that failures in software systems are often caused not by technical limitations, but by ineffective communication and incomplete requirements. Their research in requirements engineering highlighted that misunderstandings between stakeholders frequently lead to systems that fail to satisfy user needs. This issue is relevant with the university timetabling system, where students, lecturers, and administrative staff may all have different scheduling expectations and priorities.

The approach to be selected should cover almost all the stakeholders who have the most interest in the system. Admin staff and the teaching staff should be given the priority in this regard, where they should be asked about the coordination problems they face. Teachers are the stakeholders with the most interest, as they want a centralized space where admin staff can easily be asked for the change requests, and their respective students may automatically get notified on any timetable change. Students with the least interest may only be interested in the notification system where they would want to get notified for any change in schedule. The students who need to opt for electives may also want minimal clashes in their schedule.

On the other hand, all discussed stakeholder groups above are highly impacted. Delayed or no notifications for students and teachers may waste their time or may create confusion resulting in missed classes. On the other hand, it can be sometimes hectic for the admin staff when they may get a large number of change requests.

Sommerville and Sawyer (1997) suggested that effective requirement elicitation depends heavily on active stakeholder participation throughout system development. Their research summarized that involving multiple stakeholder groups helps developers gain a more complete understanding of user expectations and operational challenges. This is particularly important for the proposed project, where scheduling decisions affect different groups in different ways.

2.3. Development Approach

2.3.1. Software Development Models

Software development models provide structured approaches for planning, designing, implementing, and evaluating software systems, covering software development lifecycle. Different models are used depending on project requirements, complexity, stakeholder involvement, and the level of flexibility required during development. Selecting an appropriate development approach is an important factor influencing project success, usability, and adaptability.

Traditional software development approaches such as the Waterfall model follow a sequential process where development stages are completed one after another. Royce (1970), whose work is associated with the Waterfall approach, described software development as a structured progression through sequenced phases such as requirement analysis, design, implementation, testing, and maintenance. This approach provides clear documentation and planning beforehand, but when it comes to practical implementation, many researchers and developers argue that it becomes less effective when requirements evolve during development.

As software systems became user-centred and dynamic with time, more flexible development approaches emerged. Agile and Iterative models gained popularity as they accommodate changing requirements and continuous user feedback. Beck et al. (2001), through the Agile Manifesto, emphasized adaptability, stakeholder collaboration, and iterative improvement over rigid planning and documentation-heavy processes. This represented a shift from predictive method towards adaptive approaches that prioritize responsiveness and user involvement.

Then comes the one that I mostly prefer, the Incremental development model which also became widely adopted for systems requiring modular implementation. I prefer this the most due to its modular nature, where the developers can work in increments based on feature division. Larman and Basili (2003) discussed how iterative and incremental development approaches improve project adaptability by dividing systems into manageable functional increments while continuously refining them through feedback cycles. Their research highlighted that many successful modern software systems follow some form of iterative-incremental development due to its practical balance between structure and flexibility.

As timetable systems involve multiple stakeholders and changing communication requirements, selecting an appropriate development approach is an important task prior to starting the project. The next section further compares different development models to identify an approach that best supports flexibility, stakeholder feedback, and iterative refinement.

3.2. Comparison

One of the widely recognized traditional models is the Waterfall model, which follows a linear structure. Output of one phase becomes an input for the next phase. Royce (1970) suggested that development phases should progress systematically from requirements to maintenance one by one. While this approach provides strong documentation and clear project structure and is suitable for high-risk projects, later studies identified limitations when dealing with changing or unclear requirements. For a timetable management system, where requirements may evolve during development as stakeholder feedback is collected, the suitability of a pure sequential model is reduced significantly.

Iterative development focus on refining the system through repeated development and feedback cycles. Larman and Basili (2003) mentioned that iterative approach improves software quality by allowing problems and usability issues to be identified earlier within the lifecycle. This approach is beneficial for systems where user interaction and communication processes are difficult to fully define at the beginning of development.

Incremental development divides the system into smaller functional modules delivered over time. According to incremental approach research covered by Larman and Basili (2003), the developers can focus on manageable system components while reducing implementation risk. Features can be developed independently and integrated progressively throughout the project lifecycle. However, this approach alone may not sufficiently emphasize continuous refinement unless combined with iterative feedback practices.

Some studies also discuss the Spiral model, proposed by Boehm (1988), which combines iterative development with strong risk analysis activities. It is also cyclic nature, but the major focus is on risk analyses and mitigation in each cycle. So, it is generally considered more suitable for large-scale or high-risk industrial systems due to its complexity and management overhead.

Overall, the literature suggests that no single development model is universally suitable for all systems. Instead, the effectiveness of a development approach depends on project requirements, stakeholder involvement, and the level of adaptability required during development.

3.3. Justification of Iterative-Incremental Approach

Based on the reviewed literature, an iterative-incremental development approach is selected for the proposed timetabling support system.

The proposed system involves multiple stakeholders, including students, lecturers, and administrative staff, each having different priorities and expectations regarding timetable management and communication processes. As highlighted by Nuseibeh and Easterbrook (2000), communication issues and incomplete requirements are among the major causes of software project failure. This indicates the importance of adopting a flexible development approach.

Iterative development was considered particularly suitable because it supports continuous feedback and usability refinement. Larman and Basili (2003) emphasized that iterative approaches allow developers to identify problems early and improve systems with time based on stakeholder input.

The incremental aspect of the selected approach provides practical advantages for the project. Since the proposed system contains multiple functional areas, such as constraint submission, timetable change requests, and communication notifications, developing the prototype incrementally allows features to be implemented and evaluated in manageable stages. This will make the prototype development manageable and will ensure the availability of a working prototype, even if some of features are left by the end.

The preference for iterative-incremental hybrid approach offers more flexibility for handling evolving requirements and stakeholder feedback. It can provide a balanced combination of structure, flexibility, and stakeholder involvement. This approach aligns closely with the project's objective of developing a timetable support system that can evolve based on user feedback and evaluation findings.

2.4. Requirement Elicitation

2.4.1. Importance

Requirement elicitation is one of the most important phases within software development lifecycle, as it focuses on identifying stakeholder needs, expectations, and system requirements before starting the implementation work. Nuseibeh and Easterbrook (2000) mentioned that communication problems between stakeholders and developers are one of the primary causes of unsuccessful software systems. Their work emphasized that understanding user requirements accurately is important for developing systems that align closely with the user needs.

Within academic timetabling environments, requirement elicitation becomes particularly important due to the involvement of multiple stakeholders with different priorities. Students may focus on timetable flexibility and reduced clashes, lecturers may prioritize teaching availability and workload balance, while administrative staff are often concerned with room allocation and scheduling efficiency.

Sommerville and Sawyer (1997) suggested that successful requirement engineering depends on active stakeholder involvement and continuous communication throughout system development. Therefore, requirement elicitation should not be treated as a one-time activity. Instead, it should be considered as an iterative process where requirements evolve as stakeholders provide feedback and gain a clearer understanding of system.

For the proposed project, requirement elicitation is significant because the system aims to address communication and coordination challenges within timetable management processes. Understanding how stakeholders currently interact with timetable systems, what communication difficulties they face, and what improvements they expect is essential for designing a user-centred prototype.

2.4.2. Interviews

Interviews are among the most used qualitative requirement elicitation techniques as they allow developers and researchers to gather detailed information directly from stakeholders while also exploring experiences, frustrations, and expectations in depth.

Sommerville and Sawyer (1997) emphasized that interviews are effective for understanding complex user requirements that may not be easily captured through structured documentation alone. Research by Nuseibeh and Easterbrook (2000) also highlighted that direct communication with stakeholders improves requirement clarity and reduces misunderstandings during system development. Interviews allow follow-up questions that can also help in the clarification of responses.

Different forms of interviews may be used depending on project needs, including structured, semi-structured, and unstructured interviews. Structured interviews include predefined questions that are asked in a fixed order to all participants, semi-structured interviews include a combination of prepared questions and flexible discussion, allowing follow-up questions, and unstructured interviews are mostly open-ended discussions where conversation flows naturally.

For this project, semi-structured interviews have been conducted because they provide flexibility while still maintaining focus on predefined discussion areas. This flexibility allows participants to discuss issues that developers may not have initially considered.

However, interviews may also have certain limitations. Conducting interviews can be time-consuming, and the quality of collected data often depends on participant engagement and interviewer experience. In some cases, stakeholders may struggle to fully express or may unintentionally omit important details.

For the proposed project, interviews are considered suitable technique due to the communication-focused nature of the system. Conducting interviews with students and academic stakeholders can provide insights into timetable related communication problems, helping better understand their expectations.

2.4.3. Questionnaires and Surveys

Questionnaires and surveys are widely used quantitative requirement elicitation techniques that allow researchers to collect information from larger groups of participants efficiently. Unlike interviews, surveys are more suitable for identifying trends, patterns, and general stakeholder opinions. Sommerville and Sawyer (1997) suggested that questionnaires are useful for gathering structured feedback from multiple stakeholders while reducing the time required. Surveys also allow responses to be analyzed statistically, making them valuable for identifying common issues and summarizing stakeholder satisfaction levels.

Within user-centred systems, questionnaires are frequently used to support qualitative findings gathered through interview. Combining surveys with other elicitation methods improves requirement completeness and reduces the risk of relying on a single source of stakeholder input. This way, any questions left while interviewing stakeholders, or the ones with short answers can be asked through questionnaires.

However, they also have limitations. Responses may sometimes lack detail, and participants may misinterpret questions or provide incomplete information. Additionally, surveys generally provide limited opportunities for follow-up clarification compared to interviews.

Despite these limitations, questionnaires remain useful within academic environments due to the different group of stakeholders involved. For the proposed project, surveys can help identify common communication issues experienced by students and staff. Furthermore, nowadays, online surveys can help save a lot of time, especially in case of students, where interviewing each student can be costly.

2.4.4. Observation Techniques

Observation techniques are used within requirement elicitation to study how users interact with systems and perform tasks within their actual working environment. Unlike interviews and surveys, which rely on stakeholder explanations, observation focuses on identifying real behaviours, workflows, and communication practices directly.

Beyer and Holtzblatt (1998), through their work on contextual design, wrote that users often struggle to communicate their needs or explain workflow problems accurately. Their research suggested that observing users within real environments can reveal hidden issues and inefficiencies that may not emerge through interviews alone.

Observation is useful for identifying informal workflows, workarounds, and communication gaps that stakeholders may consider too routine or insignificant to mention explicitly. Within timetable management environments, observation may reveal how timetable

updates are currently communicated, how scheduling changes are managed, or how users respond to timetable conflicts. Myself as a student, I can enlist some of my observations throughout my Master's tenure.

However, observation can also be time intensive as for a good observation, you need to be a part of the environment for some noticeable time to gather routine experience. Moreover, user behavior may sometimes change temporarily when participants know they are being observed.

2.4.5. Hybrid Approach

Research within requirements engineering frequently suggests that no single elicitation technique is sufficient for fully understanding stakeholder needs and system requirements. Different techniques provide different forms of insight, and relying on one approach may result in incomplete or biased requirements.

As Sommerville and Sawyer (1997) recommended combining multiple elicitation methods to improve requirement quality and completeness. Moreover, Nuseibeh and Easterbrook (2000) emphasized that stakeholder communication challenges often require iterative and multi-method approaches to reduce misunderstandings and requirement gaps.

Interviews provide detailed qualitative insights, questionnaires support quantitative analysis which may not be covered in interviews, while observation reveals hidden workflow-related issues. The proposed project therefore adopts a hybrid requirement elicitation approach consisting of interviews, questionnaires, and limited observation.

The hybrid approach also aligns closely with iterative development principles, where stakeholder feedback can continuously influence system refinement throughout development and evaluation stages. By combining qualitative and quantitative methods, the project aims to develop a more complete understanding of timetable-related communication challenges and stakeholder expectations.

2.5. Prototype Evaluation

2.5.1. Importance

Prototype evaluation is also an important stage within software development lifecycle, mainly for systems that involve frequent user interaction and communication processes. Once system requirements have been gathered and an initial prototype has been developed, evaluation techniques help determine whether the proposed solution is usable, understandable, and aligned with stakeholder expectations.

Nielsen (1994) suggested that usability evaluation allows developers to identify interface and interaction problems in the early phases before final system deployment. Early evaluation is useful for reducing the chances of moving in the wrong direction that may negatively affect user satisfaction.

Evaluation should not only focus on technical functionality but also on user experience and ease of interaction. Gould and Lewis (1985) emphasized that systems should be continuously tested and refined with user involvement throughout development rather than only evaluated after completion.

For the proposed project, prototype evaluation is important because the effectiveness of the system depends on how well stakeholders can interact with communication and scheduling features. Evaluating usability and gathering feedback during development may help identify issues related to navigation, clarity, accessibility, and overall communication efficiency.

2.5.2. Usability Testing

Usability testing is about observing users when they perform tasks using a prototype or developed system. Nielsen (1994) stated that usability testing helps identify practical interaction problems that may not be visible during development alone because you view the users using the system in front of you. You may easily analyze where the users felt difficulty and what features users praised the most.

Within usability testing sessions, participants are asked to complete specific tasks while researchers or developers observe difficulties, confusion, navigation issues, or interaction delays. This allows developers to understand how users naturally interact with the system and whether system workflows are intuitive.

Research by Gould and Lewis (1985) also highlighted the importance of involving real users throughout testing and refinement activities. Their work suggested that repeated evaluation and redesign improve overall usability and user satisfaction. For me, it is similar to cooking the same recipe multiple times until the taste, balance, and presentation are refined according to user preference.

For the proposed project, usability testing can help evaluate how effectively users interact with timetable communication features such as timetable updates, request submission, and notification management.

2.5.3. System Usability Scale

The System Usability Scale (SUS) is a questionnaire-based usability evaluation method introduced by Brooke (1996). It usually consists of ten short statements that participants respond to using a Likert scale, producing a numerical usability score.

SUS is useful because it provides a simple and efficient way to measure perceived usability while requiring relatively little time from participants. SUS allows usability findings to be represented quantitatively, that makes it easier to compare overall user satisfaction levels statistically.

Brooke (1996) suggested that SUS can be applied to different types of systems regardless of the complexity or domain. Due to its simplicity, the technique has become widely adopted within usability evaluation studies and prototype testing activities.

SUS is often used alongside qualitative evaluation methods such as usability testing. While usability testing helps in identifying interaction problems, SUS provides a measurable statistic of overall system usability. Combining both approaches allows to gather both detailed qualitative feedback and supporting quantitative data.

2.5.4. Iterative Feedback and Refinement

Rather than treating evaluation as a final-stage activity, iterative feedback and refinement may reveal issues that developers did not initially notice. These may include navigation difficulties, unclear terminology, or any ineffective workflows that may result in confusion. Iterative refinement allows these problems to be addressed gradually rather than remaining unresolved until final deployment.

Larman and Basili (2003) discussed how iterative development allows developers to identify problems earlier and refine systems progressively over multiple cycles. Moreover, Gould and Lewis (1985) emphasized that usability improves significantly when users are involved during design and evaluation activities.

For timetable system, iterative evaluation may be particularly beneficial because usability issues often emerge only after stakeholders interact with prototype workflows and tasks like scheduling scenarios. Therefore, repeated feedback and refinement activities may improve both usability and communication effectiveness.

2.5.5. Evaluation Limitations

Although prototype evaluation techniques provide valuable insights, there are also certain limitations associated with these methods. One common limitation involves participant sample size. Smaller participant groups may not represent all possible user experiences or usability concerns. This is acceptable for a small-scale project or for an academic project but may become a limitation for large-scale projects.

Virzi (1992), however, suggested that even small usability testing groups can identify a significant proportion of usability problems within interactive systems. This finding supports the practicality of usability testing within smaller-scale academic projects where resources and participant availability may be limited.

Another limitation involves subjectivity within usability feedback. Different users may interpret system usability differently based on their experience, technical familiarity, or personal expectations. Observation based testing may also be influenced by participant behavior changes or mood during monitored sessions. Additionally, SUS provide overall usability scores but may not fully explain the reasons behind user dissatisfaction or confusion.

For this reason, it is recommended to combine qualitative and quantitative evaluation techniques to improve reliability and completeness of findings. Despite these limitations, combining more than one evaluation methods remains an effective approach for evaluating user-centred systems.

2.6. Ethical Considerations

2.6.1. Participant Consent and User Feedback

Ethical considerations are important when conducting research involving human participants, particularly during requirement elicitation and prototype evaluation activities. Participants involved in interviews, questionnaires, and usability testing should be informed about the purpose and how their responses will be used within the project.

The British Psychological Society (2021) highlights that informed consent is an important part of ethical research practice, ensuring that participants understand the purpose, procedures, and potential use of collected information before taking part in a study. Participants should also be given the option to withdraw from the process in any phase if they no longer wish to participate. This helps ensure that data collection activities remain transparent and ethically responsible.

For the proposed project, participants involved during requirement gathering and usability evaluation stages will be informed about the purpose of the study before providing feedback. User feedback collected during testing sessions will only be used for academic and development purposes related to the project.

2.6.2. Data Privacy and Confidentiality

Data privacy and confidentiality are important considerations when collecting stakeholder information during research activities. Requirement elicitation and usability testing may involve collecting opinions, experiences, scheduling concerns, or usability feedback from participants. Therefore, it is important to ensure that participant information is handled securely and responsibly.

The Information Commissioner's Office (ICO, 2024) states that organizations collecting personal data should ensure that information is processed lawfully, securely, and only for specified purposes. Although this project is academic in nature, similar principles remain relevant when handling participant responses and feedback data.

So, for the proposed project, collected participant data will remain confidential and will not be shared outside academic purposes. Personal identifiable information will not be disclosed within the final report.

2.7. Project Positioning

Studies such as Burke et al. (1997) and McCollum (2007) discussed approaches for improving timetable generation efficiency through automated scheduling and constraint-based heuristic algorithms. While these studies contribute towards timetable optimization, less emphasis is placed on communication efficiency and stakeholder interaction within timetable management processes.

The reviewed literature also shows that requirement elicitation and usability evaluation are considered important within software engineering. Research by Nuseibeh and Easterbrook (2000) highlighted the importance of effective stakeholder communication during requirements engineering, while Nielsen (1994) and Brooke (1996) focused on usability evaluation and user-centred testing practices.

The proposed project attempts to focus on improving stakeholder communication and interaction during timetable management processes. Rather than developing a timetable generation system, the project focuses on communication support features such as requirement collection, timetable change requests, and update handling. In addition, a hybrid requirement elicitation approach is adopted alongside iterative-incremental development and usability-focused evaluation methods. This combination allows stakeholder feedback to help system refinement continuously throughout development.

2.8. Conclusion

This literature review explored research related to timetabling systems, communication challenges, requirement elicitation techniques, prototype evaluation methods, and software development approaches.

The literature demonstrated the importance of effective requirement elicitation and continuous stakeholder involvement during software development. Combining multiple elicitation techniques helps improve requirement completeness and reduces misunderstandings. Moreover, the review highlighted the value of usability testing, iterative refinement, and user-centred evaluation approaches.

Different software development approaches were also reviewed and compared. While waterfall model provides structured development processes, the literature suggested that iterative and incremental approaches are more suitable for systems involving evolving requirements and continuous stakeholder feedback.

Overall, the reviewed literature supports the decision to adopt a hybrid requirement elicitation approach, usability-focused prototype evaluation techniques, and an iterative-incremental development methodology for the proposed timetable support system for NTU.

CHAPTER 3: IDEAS AND APPROACH

3.1. Overview

This chapter outlines the overall idea behind the proposed prototype and the approach taken to move from the problem identified in Chapter 1, and the techniques reviewed in Chapter 2, towards an actual system that can be built and evaluated. Rather than jumping straight into the implementation work, this chapter explains the reasoning behind the key decisions made before implementation began, including the development approach selected, the general concept of the prototype, the stakeholder groups the system is intended for, and the tools and technologies considered for building it.

The requirement elicitation, design, and actual development work carried out are discussed in detail in Chapter 4. This chapter focuses more on the "why" behind those decisions, setting up the reasoning that Chapter 4 then builds on.

3.2. Project Idea and Concept

As discussed in Chapter 1, the problem this project is addressing isn't the generation of the timetable itself, but rather everything that happens around it, the communication side of things. NTU already has a system in place for students and staff to view their timetables, so building yet another timetable viewer or generator wouldn't really add much value. Instead, the idea is to build a prototype that focuses purely on the interaction layer: submitting constraints, requesting changes, tracking the progress of those requests, and receiving notifications when something changes.

This idea was shaped further after conducting the first interview with a lecturer (discussed in detail in Chapter 4), which confirmed a lot of the assumptions made in the literature review. The current process relies heavily on spreadsheets, Microsoft Forms, and one-directional email communication, with no way for a requester to know the actual status of their request or the reasoning behind a decision. Based on this, the prototype concept was narrowed down to three core areas:

- **Constraint and change request submission** – allowing users to submit scheduling preferences, or request changes to existing sessions, in a structured but flexible way.
- **Progress tracking and two-way communication** – giving requesters visibility into the status of their submitted requests, and a lightweight way to ask questions or receive clarification, rather than the current "submit and wait" approach.
- **Notifications and Emails** – centralising updates so that stakeholders, emails particularly for students, are informed of any change without needing to check multiple sources.

It was also decided early on, based on scope discussed in Chapter 1, that students would primarily interact with the notification side of the system, while lecturers and the timetabling team would be the main users of the request submission and progress-tracking features. This distinction was important, because it meant the requirement elicitation activities (covered in Chapter 4) could be tailored differently depending on which stakeholder group was being approached.

3.3. Development Approach

Based on the comparison of development models carried out in the Literature Review (Section 3.2.2 and 3.2.3), an iterative-incremental approach was chosen for developing the prototype. This decision wasn't made lightly, since a few different models could technically have worked, but iterative-incremental fit the nature of this project best for a few reasons.

Firstly, requirements were not going to be fully known upfront. Since a large part of this project depends on stakeholder feedback (from lecturers, students, and the timetabling team), it made more sense to develop the system in stages, testing and refining as new information came in, rather than trying to lock down every requirement before writing any code. Larman and Basili (2003) support this reasoning, mentioning that iterative approaches allow problems to surface earlier in the process, rather than only being discovered once a system is "finished."

Secondly, the incremental aspect made the project more manageable in terms of scope and risk. Rather than treating the prototype as a single large deliverable, it was broken down into functional increments: constraint submission, change request handling, and notifications. This meant that even if one feature ended up more complex than expected or had to be scaled back, the rest of the prototype could still be delivered as a usable system.

This approach also lines up naturally with the usability evaluation techniques discussed in Chapter 2 (usability testing and SUS), since each increment can be tested and evaluated separately before moving to the next, rather than saving all evaluation until the very end.

3.4. Proposed System Overview

At a high level, the proposed prototype consists of the following core components, based on the concept described in Section 3.2:

- A **request submission interface**, where lecturers can submit new constraint preferences or change requests for existing sessions.
- A **request tracking / status view**, allowing users to see the current stage of their submitted requests (e.g. pending, in progress, actioned), addressing one of the main pain points identified during the lecturer interview.
- A **notification system**, primarily aimed at students, that centralises updates relating to session changes, cancellations, or new/rescheduled sessions, rather than relying on scattered emails or manual timetable checking.

The exact architecture, workflows, and interface design for these components are worked through in detail in Chapter 4, once the requirement elicitation activities had been carried out with all three stakeholder groups.

3.5. Tools and Technologies Considered

Following further discussion and decision-making around scope and feasibility, the following technology stack was finalised for prototype development:

Table 1. Tools and Technologies Considered

Layer	Technology	Justification
Backend	Java (Spring Boot)	• Prior development experience • Strong support for REST APIs • Role-based authentication (Spring Security) • ORM mapping via Spring Data JPA
Database	MySQL	• The system's data (users, roles, modules, sessions, rooms, requests, comments, statuses) is highly relational, with clear foreign-key relationships and integrity constraints
Frontend	React	• Component-based structure suits the role-specific views required (Lecturer vs Timetabling Team) • Consumes the backend's REST APIs directly, keeping frontend and backend development independent
API Communication	REST (JSON)	• Separation of concerns between client and server • Allows each increment's backend functionality to be tested independently of the frontend

A relational database (MySQL) was chosen over a NoSQL alternative such as MongoDB, as the project's data does not require the flexible, loosely structured document model NoSQL databases are typically suited to. Instead, the data involved is naturally relational: requests reference specific modules, sessions, and users, and validating a request against the live timetable (FR4) requires querying for overlaps in time and room allocation, operation SQL databases are well equipped to handle through structured joins and constraints.

3.6. Chapter Summary

This chapter has outlined the reasoning behind the core project idea, the decision to adopt an iterative-incremental development approach, and a high-level view of what the prototype is intended to consist of. These decisions were informed both by the literature reviewed in Chapter 2 and by early conversations with stakeholders. Chapter 4 now moves into the practical side of the project, covering the requirement elicitation process in detail, the design decisions made, and the details for incremental development of the prototype.

CHAPTER 4: IMPLEMENTATION AND INVESTIGATION

4.1. Requirements Elicitation

4.1.1. Process Overview

As outlined in Chapter 3, a hybrid requirement elicitation approach was adopted for this project, combining semi-structured interviews and online questionnaires, in line with the recommendation from Sommerville and Sawyer (1997) that combining multiple elicitation methods improves requirement completeness and reduces the risk of relying on a single source of stakeholder input.

The method used for each stakeholder group was chosen based on the size and nature of that group, rather than applying the same technique everywhere.

- A lecturer and a member of the timetabling team were approached individually through semi-structured interviews, since these stakeholders have detailed, first-hand experience of the current request-handling process, and open-ended discussion allowed follow-up questions to explore specific issues as they came up.

- Students, on the other hand, were approached through an online Microsoft Forms questionnaire distributed to a wider group. Since students in this project were only expected to interact with the notification side of the system (as scoped in Chapter 3), a shorter, largely closed-question survey was considered more appropriate than individual interviews, both to respect participants' time and to gather more directly comparable, quantifiable responses.

- A short questionnaire was also distributed to two members of the timetabling team as a supplementary method alongside the interview, to try and gather a wider set of views from that stakeholder group, though as discussed in Section 4.1.3.

All participants were provided with a Participant Agreement prior to taking part, outlining the purpose of the research, voluntary nature of participation, and how their responses would be used and kept confidential, in line with the ethical considerations discussed in the Literature Review (Section 6 / Appendix).

4.1.2. Interview Findings – Academic Staff

A semi-structured interview was conducted with a lecturer to understand how academic staff currently interact with the timetabling system and what they would want from an improved prototype.

The participant described interacting with the timetable primarily through Outlook calendar on a daily basis during term time, occasionally supplementing this with the timetabling website directly when quick, accurate information was needed. Their main timetabling request happens once a year per module (up to three times, given they lead three modules), while general change requests happen more frequently, around 25 times a year, with no official limit on how many change requests can be submitted.

Several inefficiencies in the current process were raised. Initial-year requests are submitted through a large, duplicate-heavy spreadsheet template, described as error-prone and difficult to fill in concisely. Communication after submission is one-directional. Requests go to the timetabling team, but there's no dialogue back, so if the resulting timetable doesn't match what was requested, there's no way to tell whether this was a deliberate constraint conflict or an accidental omission. The current system also doesn't distinguish between hard requirements and softer preferences; everything is submitted as a flat request with no way to indicate flexibility.

Change requests during term time go through a Microsoft Form, which the participant felt was designed around single-session changes (e.g. moving one session to another day) rather than the kind of year-long preference requests or multi-module/multi-staff requests that sometimes need to be made. It was also raised that students currently can't report timetable clashes or issues directly, they have to go through their course leader or a staff member first, adding extra steps to a process that's already slow. A further issue raised was that the form doesn't check against the live timetable (for example, the fields that ask for the new schedule slot), meaning users have to look for the details by themselves using other applications.

When asked what they'd want to see prioritised in an improved system, the main points raised were:

- concise, low-effort data entry (auto-populating information where possible rather than requiring re-typing)
- visibility into the progress and status of a submitted request
- a two-way communication channel for asking questions or getting clarification
- flexible categorisation of requests
- validation against the live timetable so that only genuinely available sessions/rooms/times can be selected.

It was also made clear that any categorisation or structure shouldn't become overly rigid, the system needs to remain flexible enough to accommodate less common or more complex requests that don't fit neatly into predefined fields.

4.1.3. Interview/Survey Findings – Timetabling Team

The timetabling team survey received two responses. The first respondent currently handles between 6 and 10 change requests in a typical week, while the second handles a noticeably higher volume, between 11 and 20 requests weekly. Both respondents agreed that some proportion of requests they receive require back-and-forth clarification before they can be processed, though the reasons given for delays differed slightly: the first respondent pointed to conflicts with the existing timetable and ambiguous or unclear requests, while the second pointed specifically to missing information.

Views on the current Microsoft Form were also mixed. The first respondent felt it already captures the information needed to process requests efficiently, while the second felt it could be improved, a more moderate stance, but one that still leans towards dissatisfaction, and is closer to the concerns raised independently in the lecturer interview (Section 4.1.2).

The two respondents also described different approaches to communicating outcomes back to requesters. The first respondent indicated that requesters are expected to check the live timetable themselves to see if a change has gone through, whereas the second described a more proactive process, where the departmental timetabling team emails lecturers directly to confirm that a request has been put forward. This inconsistency is

itself a useful finding, it suggests that outcome communication currently depends on individual practice rather than a standardised process.

The disagreement in this round of findings is internal to the timetabling team itself, rather than between the timetabling team and the lecturer. On a dashboard-style view of pending/in-progress/completed requests, the first respondent rated this as not useful, while the second rated it as extremely useful. The same pattern held for categorising incoming requests by type, urgency, or approval needed, the first respondent rated this as not important, while the second rated it as very important.

Both respondents agreed on the remaining points. Both felt that automatically checking requests against the live timetable (e.g. flagging unavailable rooms/times) would help, both were comfortable with requesters being able to view the status of their own request without contacting the team directly, and both did not support students submitting requests or issues directly, preferring that this continue to go through academic staff.

4.1.4. Survey Findings – Students

The student survey received six responses. Five of the six respondents reported having experienced timetable clashes or scheduling conflicts before, with the most commonly cited reason being a late timetable change that wasn't communicated in time. The same five respondents also reported having missed or received delayed timetable updates, most commonly rarely (once or twice a semester), with one respondent reporting this happens occasionally (once a month).

When asked how quickly they usually receive timetable change notifications, most respondents indicated they only find out by manually checking their calendar, rather than being notified directly, with two respondents saying this happens "sometimes" and one respondent reporting they never receive timely notifications at all. This reinforces one of the core assumptions behind the project's scope: that students are currently left to check for updates themselves rather than being proactively informed.

Every respondent said they would prefer to receive timetable updates in real-time, as soon as a change is made. Preferred notification channels were split fairly evenly between email and mobile app notifications, with three respondents selecting each. In terms of what changes matter most, time changes were flagged by nearly all respondents, followed by cancelled sessions, new/rescheduled sessions, and room changes, suggesting students want to be notified of most types of change rather than one specific category.

Most respondents considered at least 24 hours of advance notice to be acceptable, with one respondent indicating same-day notice would be fine. Interest in having a single, centralised place to view all timetable-related notifications was high, four respondents said they would be very interested, and the remaining two said they would be somewhat interested, with no respondent expressing disinterest.

4.1.5. Synthesis of Findings

Bringing the interview and survey findings together below:

Students

- Strongest, most consistent group across all responses.
- Every respondent wanted real-time updates.
- Interest in a single, centralised place for notifications, rather than manually checking a calendar.
- Directly supports the project's decision (Chapter 3) to scope student involvement around the notifications.

Lecturer vs Timetabling Team — points of agreement

- Both agreed live-data validation of requests (checking availability before submission) would be valuable.
- Both agreed requesters should be able to view their own request status without contacting the timetabling team directly.

Internal variation within the Timetabling Team

- The two timetabling team respondents disagreed with each other on both the dashboard view and request categorisation, one rated both as not useful/not important, the other rated both as extremely useful/very important.
- The second respondent's views align closely with the lecturer's original request for these features, suggesting the earlier apparent "disagreement" between lecturer and timetabling team may say more about individual working style than a team-wide rejection of these features.

Limitation to note

- The stakeholder count was kept low due to the limited availability of staff and students during the summer period.
- Further responses or a follow-up interview would strengthen requirements tied specifically to the timetabling team's workflow.

Overall priorities supported by these findings

1. A centralised, real-time notification system for students.
2. Progress tracking and live-data validation for lecturers submitting requests.
3. A dashboard and categorisation feature for the timetabling team.

4.1.6. Requirements Table

Based on the findings from Sections 4.1.2–4.1.5, the following table enlists the functional and non-functional requirements identified for the prototype. Each requirement is assigned an ID, mapped to its source (Lecturer, Timetabling Team, Student, or a project-level decision made by me based on scope/literature), given a priority using **the MoSCoW** method (Must, Should, Could, Won't), and assigned to the increment it belongs to, based on the three functional modules defined in the Project Plan

- **Increment 0** – Timetable Viewer (add-on from my side) and All about Roles involved (Admin, Lecturer, Timetabling Team, Student)
- **Increment 1** – Constraint Submission
- **Increment 2** – Change Request Handling
- **Increment 3** – Two-way Communication
- **Increment 4a** - Notifications & Emails
- **Increment 4b** – Activity Logs

Increment 0 was added independently of the stakeholder findings, as a baseline requirement needed to support the other increments, since users need a way to view the existing (mock) timetable before they can meaningfully submit constraints or request changes against it.

Increment 0 was expanded during development to include an Admin role, which was not originally raised but was added independently as a practical necessity, as someone needs to create accounts, manage courses/modules, and administer the system before the other roles can meaningfully use it.

Notably, students were also given full login accounts as part of Increment 0. This decision was made so students could be properly identified and linked to their courses/modules, allowing accurate, targeted notifications and activity logs, rather than relying on a public, unauthenticated page.

Where a requirement received mixed or conflicting input across stakeholder groups, this has been reflected by assigning a lower priority than the strength of a single stakeholder's request might otherwise suggest, pending further validation.

Table 2. Functional Requirements

ID	Requirement	Source	Priority	Increment	Depends On
FR0a	The system supports four distinct roles (Admin, Lecturer, Timetabling Team, Student), each logging in with role-appropriate access and views	Myself	Must	0	–
FR0b	Users can view the current mock timetable, including sessions, rooms, and times, within the web application	Myself	Must	0	FR0a
FR0c	Admin can manage system users (create/edit, assign roles, mark leavers/alumni), courses, and modules, including creating student accounts and assigning them to courses	Myself	Must	0	FR0a

FR1	Users can submit timetabling constraint preferences ahead of the main annual timetable being created	Lecturer	Must	1	FR0b
FR2	Users can mark a submitted constraint as either a firm requirement or a flexible preference	Lecturer	Should	1	FR1
FR3	Users can submit a change request against an existing, live timetable session	Lecturer	Must	2	FR0b
FR4	Change requests are validated against live timetable data, only allowing selection of genuinely available rooms, times, or sessions	Lecturer, TT	Must	2	FR3
FR5	Users can view the current status of their submitted request (e.g. pending, in progress, actioned)	Lecturer, TT	Must	2	FR3
FR6	The system records a reason/comment when a request is actioned or rejected, visible to the requester	Lecturer	Should	2	FR5
FR8	Requests can be categorised by type	Lecturer, TT	Should	2	FR3
FR9	A single request can be linked to multiple modules or staff members where relevant	Lecturer (After Evaluation 01)	Could	2	FR3
FR12	Users can view a dashboard with necessary information / summary.	Lecturer, TT	Should	2	FR5
FR13	Students do not submit constraint/change requests directly; requests continue to be routed through academic staff.	TT	Must	2	FR0a
FR7	A two-way communication thread is attached to each request, allowing lecturers and the timetabling team to exchange questions/clarifications	Lecturer	Should	3	FR3
FR10	Users receive notifications (in-app or email) for relevant changes, such as request status updates and session/timetable changes	Lecturer, TT, Std	Must	4a	FR5
FR11	Users can view an activity log of relevant changes, scoped by role: • Timetabling Team see all the activity	Lecturer, TT, Std, Myself	Should	4b	FR0a, FR5

	• Lecturers see their own requests • Students see changes affecting their own course				

Table 3. Non-Functional Requirements

ID	Requirement	Source	Priority	Increment	Depends On
NFR1	The system should minimise manual data entry through auto-population and dropdowns where possible	Lecturer	Should	1, 2	–
NFR2	The system should remain flexible and avoid overly rigid/prescriptive fields, allowing free text where structured input isn't sufficient	Lecturer	Must	1, 2	–
NFR3	The interface should be simple, intuitive, and require minimal effort to learn and use	Myself (Lit Review)	Must	All	–
NFR4	The prototype does not require institutional single sign-on; a simplified login/authentication can be used for prototype purposes	The Supervisor and I	Won't (for prototype)	–	–
NFR5	All stakeholder data collected during requirement elicitation and evaluation must be stored securely and used only for academic purposes	Ethical Considerations	Must	All	–
NFR6	The system must enforce role-based access control, ensuring each of the four roles can only access functionality and data relevant to their role	Myself	Must	0	FR0a

4.1.7. Proposed System Components

Based on the finalised requirements, the prototype is structured around a single, login-based web application supporting four distinct roles:

Web Application

- **Admin** — manages users (including creating student and staff accounts), courses, and modules, and administers the system before other roles can meaningfully use it (FR0a–FR0c).

- **Lecturer** — logs in with an account created by an Admin, views the dashboard with summarized information, views the timetable, submits constraint preferences (Increment 1) and change requests (Increment 2), communicates via a two-way thread with the Timetabling Team (Increment 3), and views the status, reasoning, and activity log relating to their own requests (FR1–FR9, FR11).

- **Timetabling Team** — logs in with an account created by an Admin, views the dashboard with summarized information, views the timetable, communicates via the two-way thread, and can view a complete activity log across the system (FR4, FR5, FR12, FR11).

- **Student** — logs in with an account created by an Admin and linked to their course, and has access limited to viewing the timetable, receiving notifications, and viewing an activity log scoped to their own course's changes (FR0a, FR10, FR11, FR13). Students do not submit constraint or change requests directly.

Mock Timetable Database
A self-seeded dataset representing sessions, rooms, times, and modules, standing in for NTU's live timetable system. This acts as the source of truth used to validate incoming requests (FR4) and to trigger notifications whenever a change is actioned (FR10). This approach was adopted following discussion with the project supervisor, since live access to NTU's actual timetable data was not available for this project, and a self-managed dataset allows the prototype's core functionality to still be demonstrated meaningfully.

Student-Facing Layer (no login required)

Automatic email notifications sent whenever a relevant change is actioned (FR10).

4.1.7. Use Case Diagram

Based on the finalised requirements (Section 4.1.6) and the four roles established during development: Admin, Lecturer, Timetabling Team, and Student. Each actor has a distinct set of interactions with the system, reflecting the role-based access decided on during Increment 0.

The Admin actor is primarily responsible for setting up and maintaining the system itself, managing user accounts (including creating student and staff accounts), and managing courses and modules. This actor doesn't interact with the day-to-day request workflow but enables the other three roles to use the system meaningfully.

The Lecturer actor is the primary requester within the system, submitting constraint preferences ahead of the annual timetable, submitting change requests against existing sessions, tracking the status of their requests, and communicating with the Timetabling Team where clarification is needed.

The Timetabling Team actor is the primary processor of requests, viewing a dashboard of pending/in-progress/completed requests, validating and actioning requests against the live (mock) timetable, recording a reason for decisions made, and communicating back with lecturers where needed.

The Student actor has the most limited set of interactions, reflecting the project's scope decision to keep students focused on visibility rather than direct system involvement. Students can log in, view the timetable, receive notifications relating to their course, and view an activity log of relevant changes, but do not submit requests directly.

The table below summarises which use cases apply to which actor:

Table 4. Use Cases

Use Case	Admin	Lecturer	Timetabling Team	Student
Login	✓	✓	✓	✓
View Timetable	✓	✓	✓	✓
View Dashboard	✓	✓	✓	✓
Manage Users	✓			
Manage Courses & Modules	✓			
Submit Constraint Preference		✓		
Submit Change Request		✓		
View Request Status		✓	✓	
Action Request			✓	
Exchange Messages on Request		✓	✓	
Receive Notifications		✓	✓	✓
View Activity Log		✓	✓	✓
Receive Emails on Updates				✓

The use case diagram is presented across three separate figures rather than as a single combined diagram, for readability. With four actors and fifteen use cases, a single diagram became visually cluttered and difficult to follow, particularly around the relationships between related use cases *(added in Appendix)*.

Splitting the diagram by functional grouping, mirroring the increment structure used during development (Access & Setup, Constraint & Change Requests, and Communication & Awareness), keeps each figure focused on a coherent set of interactions, and ensures that only the actors actually relevant to that group are shown. This approach also makes it easier to trace a specific requirement or increment back to its corresponding use cases, since each figure aligns closely with a distinct phase of the system's development.

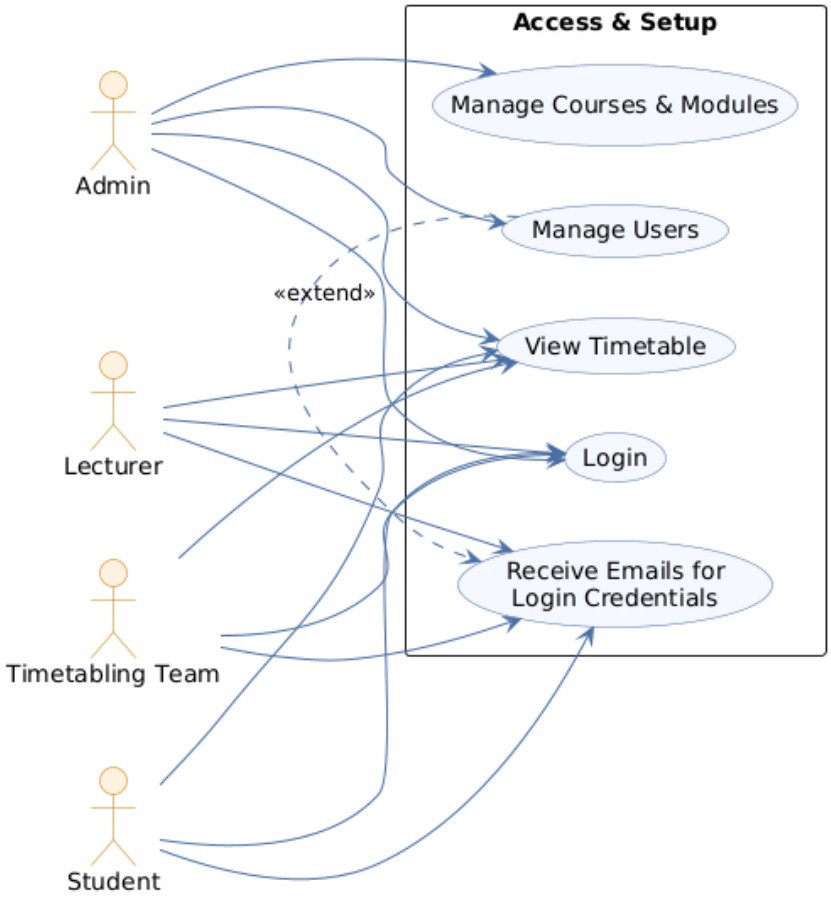

Figure 1. Use Case Diagram - Access and Setup

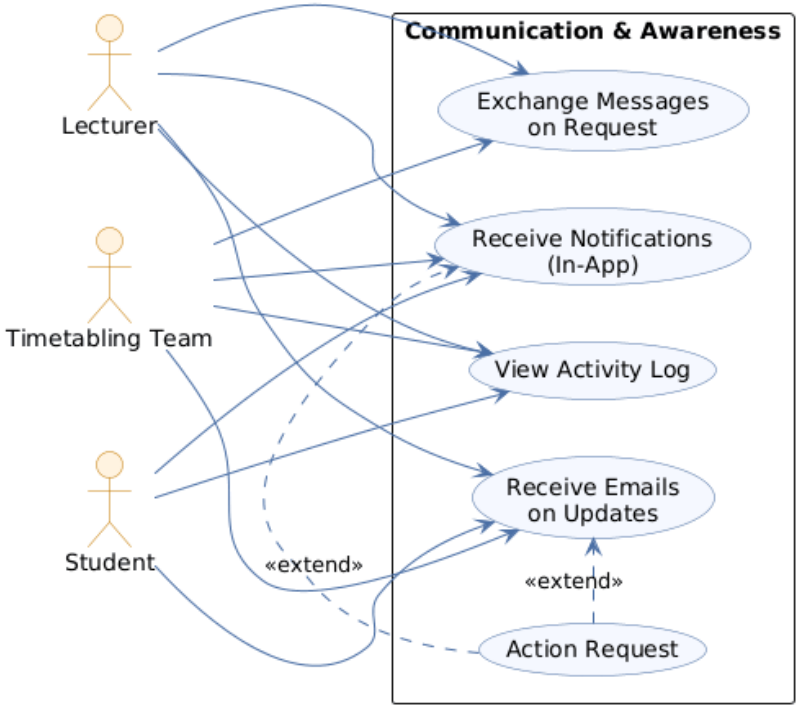

Figure 2. Use Case Diagram - Communication and Awareness

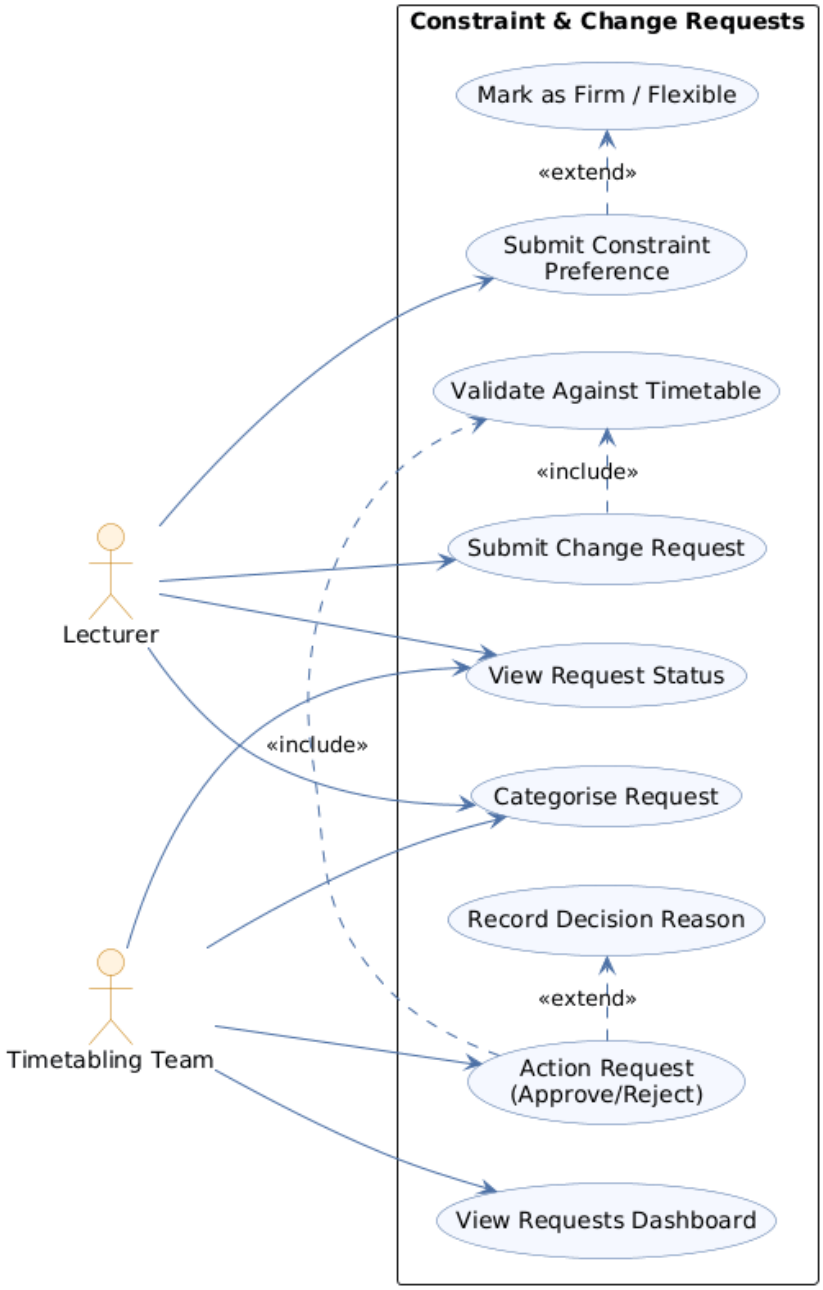

Figure 3. Use Case Diagram - Constraint and Change Requests

4.2. Design

4.2.1. System Architecture

The prototype follows a standard three-tier client-server architecture, in line with the technology stack finalised in Section 3.5:

- **Frontend (React) |** an application consuming the backend via REST APIs, with role-specific views rendered based on the logged-in user's role.
- **Backend (Java, Spring Boot) |** exposes REST endpoints for all the required services including authentication, timetable data, requests, notifications, and activity logs; and handles business logic such as request validation and role-based access control.
- **Database (MySQL) |** stores all the data, including users, courses, modules, sessions, requests, messages, notifications, and activity logs.

This separation keeps each layer independently testable and allows backend functionality to be verified (e.g. via Postman) before the corresponding frontend view is built, which suited the iterative-incremental approach adopted.

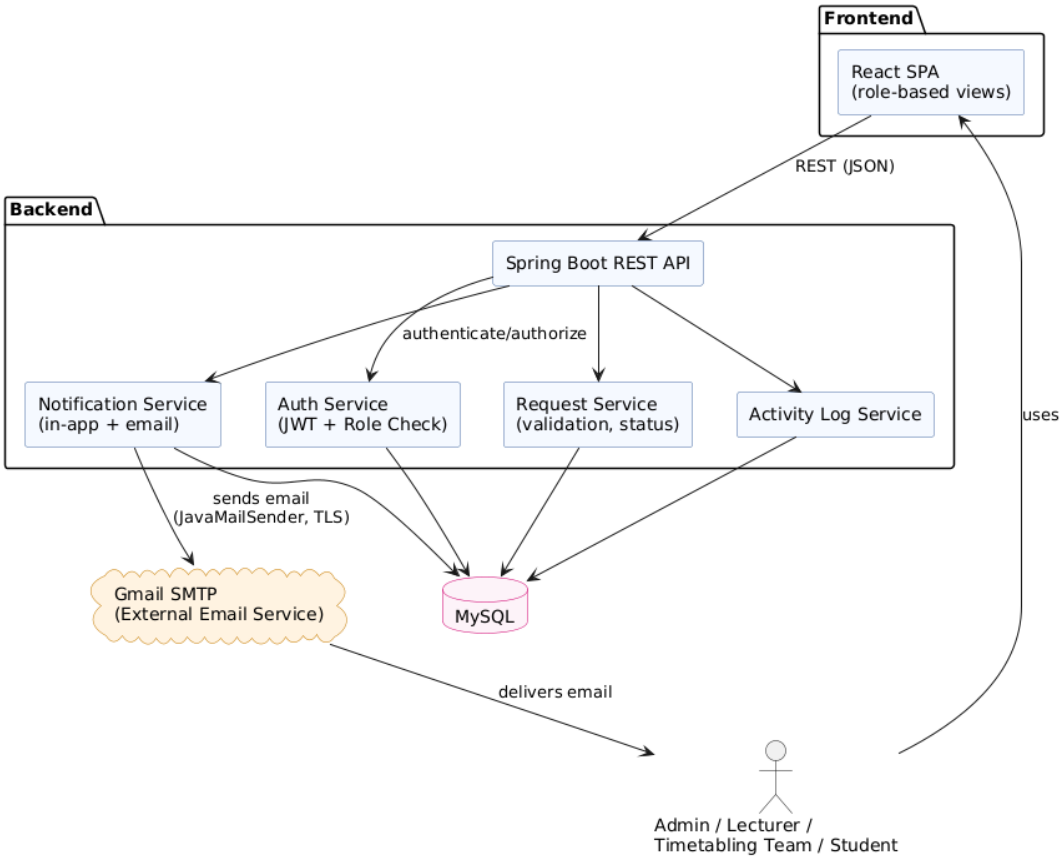

Figure 4. System Architecture Model

4.2.2. Database Design

The database schema reflects the relational nature of the system's data, discussed in Section 3.5. Key entities include:

- **users |** stores login credentials, role (Admin, Lecturer, Timetabling Team, Student), account status, and, for students, a link to their enrolled course and lab/seminar group.

- **courses, modules, rooms & timetable_sessions |** represent the mock timetable data, with sessions linked to a module, room, and lecturer. Week-varying or partially cancelled sessions are supported through supporting tables (session_overrides, session_cancelled_weeks), rather than duplicating session rows.

- **Requests** | a single, unified table covering both constraint submissions and change requests, distinguished by a type field (CONSTRAINT/CHANGE). This consolidated design avoids duplicating the shared fields (requester, status, category, reason) across two separate tables, while several supporting tables (request_weeks, request_rooms, request_groups, request_modules, request_merge_sessions, request_unavailable_days) capture more detailed, optional fields specific to certain request categories.

- **request_comments** | supports the two-way communication thread attached to a request (Increment 3), with optional file attachments stored via comment_attachments.

- **Notifications** | stores in-app notification records, generated when a request status changes or a session is updated (Increment 4a).

- **email_log |** records outgoing emails sent by the system (e.g. login credentials, request status updates), kept separate from notifications since it tracks a different channel and outcome.

- **activity_log |** a record of relevant system events, scoped by role when displayed (Increment 4b). Kept distinct from notifications, since a log is a comprehensive historical record, while notifications are personal and dismissible.

The diagram below shows the system's core entities and their key relationships at a conceptual level.

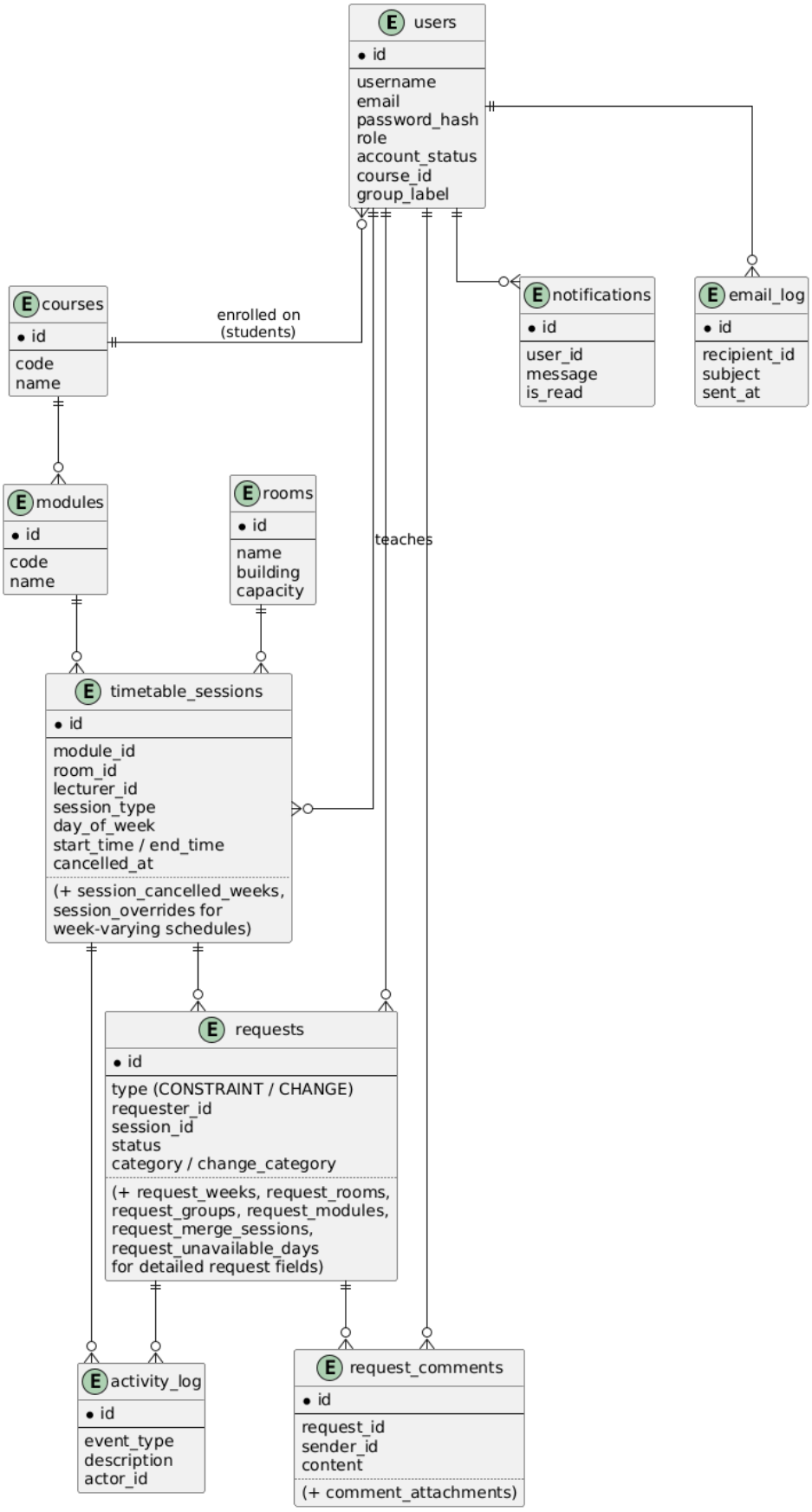

Figure 5. Entity Relationship Diagram

4.2.3. Authentication and Role-Based Access

All four roles authenticate through the same login mechanism, using a username and password issued when an account is created by an Admin (FR0a, FR0c). On successful login, the backend issues a JWT (JSON Web Token), which the frontend then includes on all subsequent requests. The backend validates this token and checks the user's role before granting access to any role-specific endpoint or data, satisfying NFR6 (role-based access control).

4.2.4. UI/UX Considerations

In line with NFR2 and NFR3, the interface was kept simple and role-focused, each role only sees navigation options and screens relevant to their own use cases (Section 4.1.7), rather than a single generic dashboard for everyone.

The interface follows a consistent visual identity built around NTU's brand colour, a pink/fuchsia accent, applied to headings, active navigation states, and key call-to-action elements, while the rest of the interface uses a clean, neutral palette of white surfaces, light grey borders, and muted secondary text to keep the pink accent purposeful rather than overused.

This same visual language was carried through consistently across both the web application and the automated email notifications (Increment 4a), so that emails are immediately recognisable as coming from the system rather than reading as a generic, unbranded message. Overall, the design prioritises clarity and consistency over visual complexity, in keeping with NFR3's requirement that the interface remain simple and require minimal effort to learn.

4.2.5. Wireframes

Two of the early wireframes are shown below, illustrating the intended layout and field structure for two of the system's core screens before full development began.

Figure 6 below presents a wireframe that shows the constraint submission screen, split into two tabs reflecting the two constraint types captured in the data model (Section 4.2.2), Module-based and Personal (FR1). The layout was kept to a single, uncluttered form rather than splitting fields across multiple steps, in line with NFR1 and NFR3 (minimal effort, simple to use), and closely informed by the lecturer's own criticism of the existing spreadsheet-based process being "duplicate-heavy" and hard to fill in concisely (Section 4.1.2).

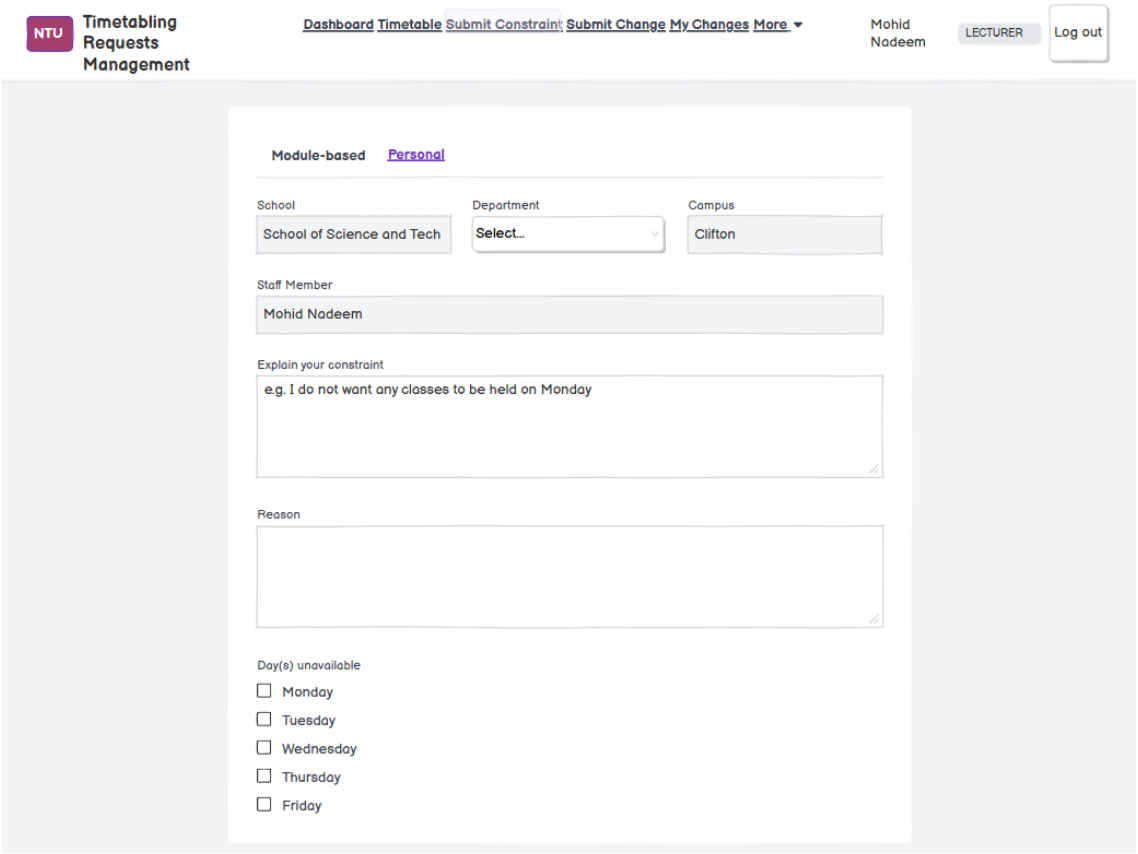

Figure 6. Wireframe - Submit Constraint (Lecturer)

Figure 7 below presents a wireframe that shows the Timetabling Team's central view of submitted change requests, directly addressing FR12 (a dashboard/queue of pending requests) and FR8 (request categorisation). This structure was shaped by the second timetabling team respondent's strong endorsement of both a dashboard view and categorisation (Section 4.1.3), while the filter row was added specifically so the team could manage a growing volume of requests without needing to scroll through an undifferentiated list.

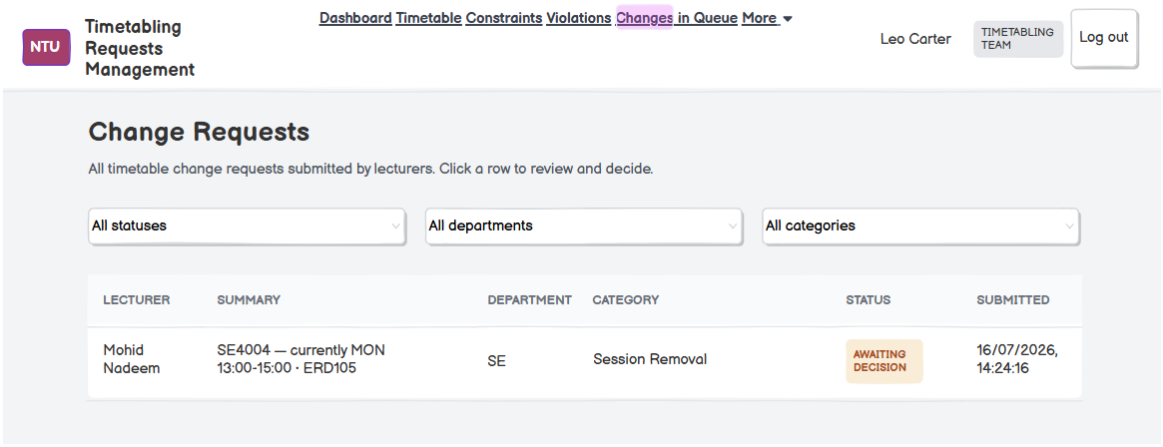

Figure 7. Wireframe - Change Requests Queue (Timetabling Team)

4.3. Development

4.3.1. Development Approach

The prototype was built incrementally, in line with the iterative-incremental approach justified in Chapter 3, with each increment corresponding to a functional slice of the requirements table (Section 4.1.6):

Table 5. Incremental Development

Increment	Focus	Key Requirements
0	Roles, login, timetable viewer, Admin (users/courses/modules)	FR0a, FR0b, FR0c, NFR6
1	Constraint submission	FR1, FR2
2	Change request handling	FR3–FR6, FR8, FR9, FR12, FR13
3	Two-way communication	FR7
4a	Notifications & emails	FR10
4b	Activity logs	FR11

Each increment was implemented backend-first (entity/schema, then REST endpoints), followed by the corresponding frontend views, and manually tested before moving to the next increment, consistent with the incremental testing approach discussed in the Project Plan (Section 3, Task 9).

4.3.2. Key Implementation Notes

Roles, login, and the Timetable Viewer (Increment 0) established the foundation the rest of the system builds on. Authentication uses JWT, with accounts provisioned by an Admin rather than self-registration, since the system sits within a single organisation (NTU) and not for the public. All seeded accounts are created with a temporary password and must_change_password set to true, forcing a password reset on first login. The shared Timetable Viewer reads directly from the same seeded session data used for request validation (FR4), giving every role a consistent, common view of the mock timetable before submitting or actioning any request.

Constraint submission (Increment 1) includes two kinds of constraints:

- **Module-based constraints** (e.g. preferred room layout, required features such as step-free access or a fixed podium, software needs, lecture capture) apply to a specific module,

- while **Personal-based constraints** (e.g. unavailable dates/times) apply to the individual lecturer regardless of module. Both share the same underlying requests table and firm/flexible distinction (FR2), keeping the submission flow consistent even though the fields relevant to each differ.

Change request handling (Increment 2) is built around a fixed set of change categories, including Session Time, Clashes, Room Type, Additional Session, Room Booking, Student Allocation, Staff Change, Session Date, Session Removal, and Merge Sessions/Groups. This category-first design was chosen based on the lecturer interview and the second timetabling team respondent's feedback (Section 4.1.3–4.1.5), both of whom supported structured categorisation (FR8). Certain categories trigger category-specific fields, for example, a Clashes request references a second, conflicting session, while a Staff Change request captures a preferred replacement lecturer, rather than forcing every category through the same generic set of fields, unlike current method (request via Microsoft Forms).

Validation against the mock timetable (FR4) is enforced server-side, requests referencing an unavailable room/time/session are rejected before being saved, rather than relying on frontend checks alone.

Notifications (Increment 4a) are triggered at specific event points in the backend (e.g. session updated, request status changed), firing both an in-app notification and an email.

Role-scoped activity logs (Increment 4b): the Timetabling Team can see all activity, Lecturers see activity relating to their own requests, and Students see only session-level changes affecting their own course, this filtering is applied server-side.

Development was carried out using **Git for version control**, in line with the mitigation strategy for data loss/file corruption identified in the Project Plan (Section 5.1).

4.3.3. Testing During Development

Each increment was tested manually as it was completed, checking core functionality before moving to the next increment, consistent with the project's iterative-incremental approach (Chapter 3).

Beyond this internal functional testing, evaluation was also carried out with stakeholders throughout development, rather than being left entirely until the end. The lecturer participant was involved incrementally, reviewing and giving informal feedback on the constraint submission and change request features as they were developed, allowing adjustments to be made early rather than waiting for a single evaluation session at the end.

A more formal usability testing and SUS evaluation session was conducted once with a timetabling team member, closer to project completion, to assess the completed request-handling and dashboard features as a whole.

Separately, the email notification component (Increment 4a) was tested with student participants, checking that notifications were received correctly, arrived promptly, and were clear and relevant, since students will be mostly interacting with the system through this channel.

This combination of incremental developer testing, ongoing lecturer feedback, a formal end-stage SUS evaluation with the timetabling team, and targeted student testing of the email channel, is discussed in full in Chapter 5, where the usability testing and SUS results are presented and analysed.

4.4. Developed Prototype System – Final Screens

This section presents the final, developed screens of the prototype, organised by role. Where a screen or feature is shared across multiple roles (e.g. login), it is described once rather than repeated.

4.4.1. Shared Screens

- **Login:** shared login screen for all four roles, with forced password reset on first login (FR0a, FR0c).

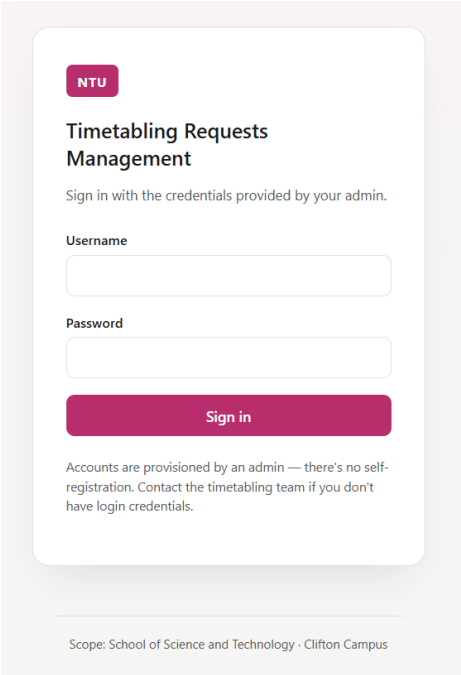

Figure 9. Screen 01 - Login

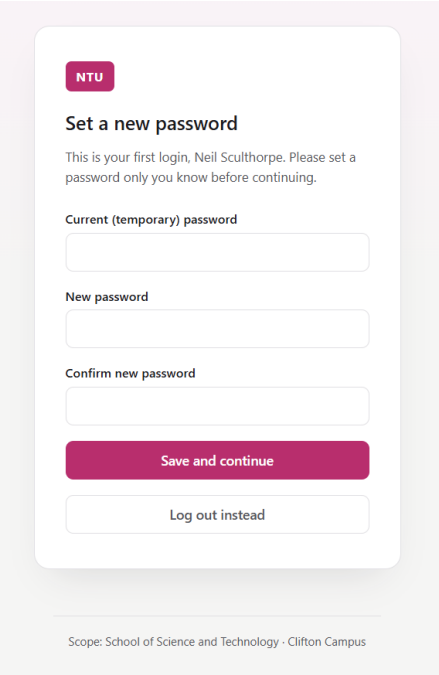

Figure 8. Screen 02 – Set Password

- **Notifications (bell/dropdown):** in-app notification list, shared across Lecturer, Timetabling Team, and Student views (FR10).

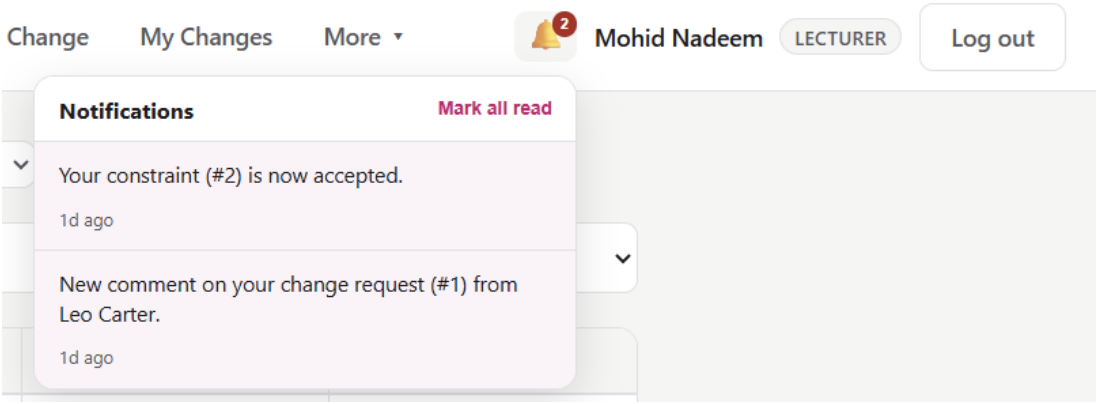

Figure 10. Notification Bell

- **Timetable Viewer:** read-only view of the current mock timetable, shared across all roles, forming the baseline data used throughout the system (FR0b).

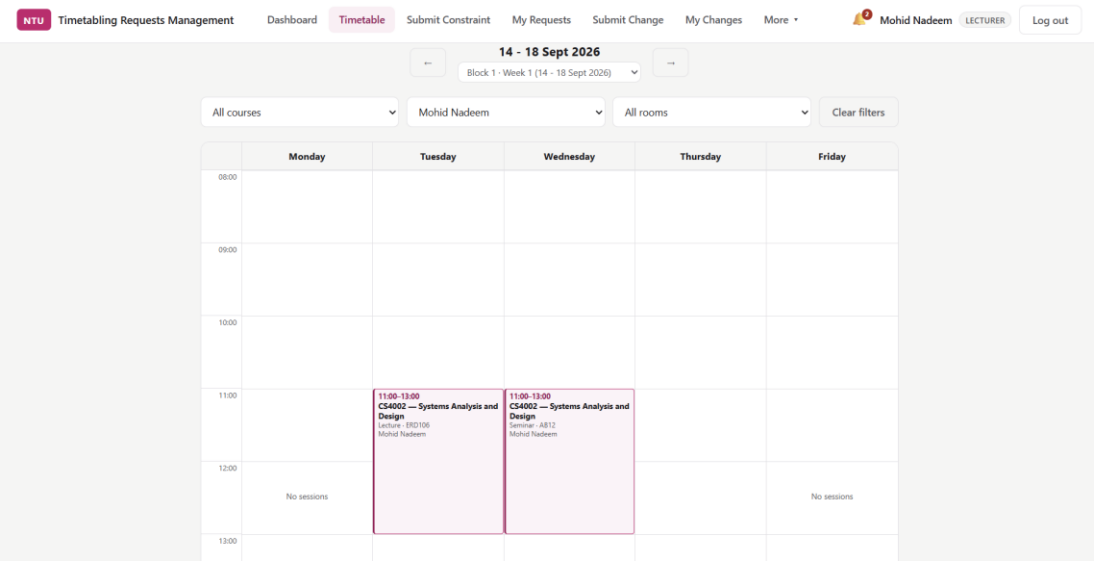

Figure 11. Screen 03 - View Timetable | Filtered

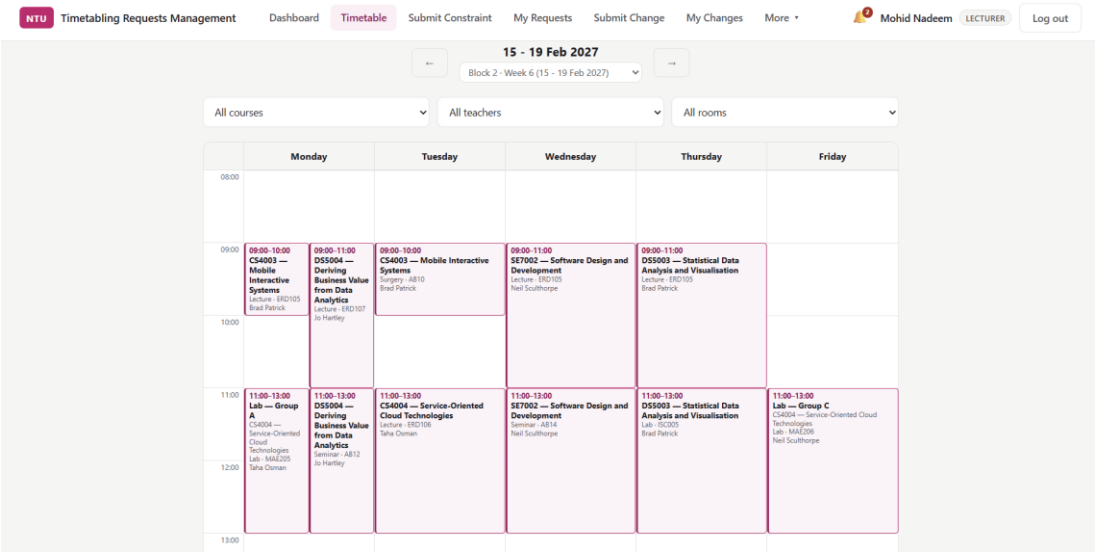

Figure 12. Screen 03 - View Timetable | No Filters

- **Activity Log:** role-scoped log view; layout is shared, but content differs by role (Timetabling Team see all activity, Lecturers see their own, Students see their course's) (FR11).

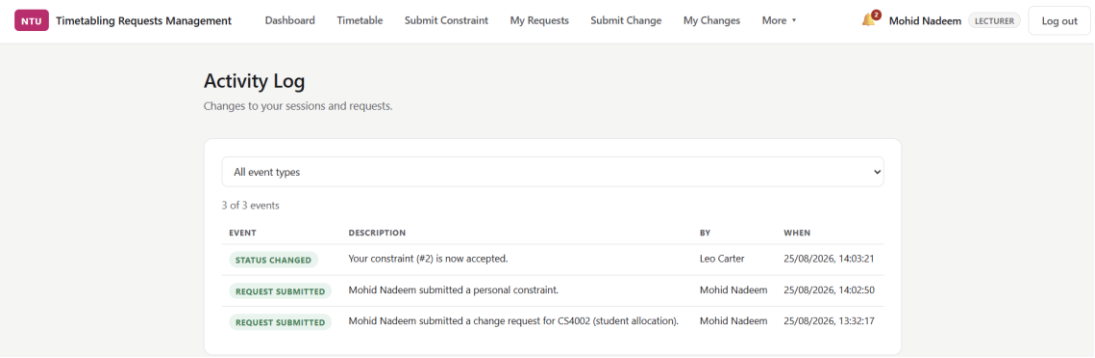

Figure 13. Screen 04 - Activity Log (Lecturer Side)

4.4.2. Admin Screens

- **Admin Dashboard** (summary cards: users, courses, modules, leavers/alumni)

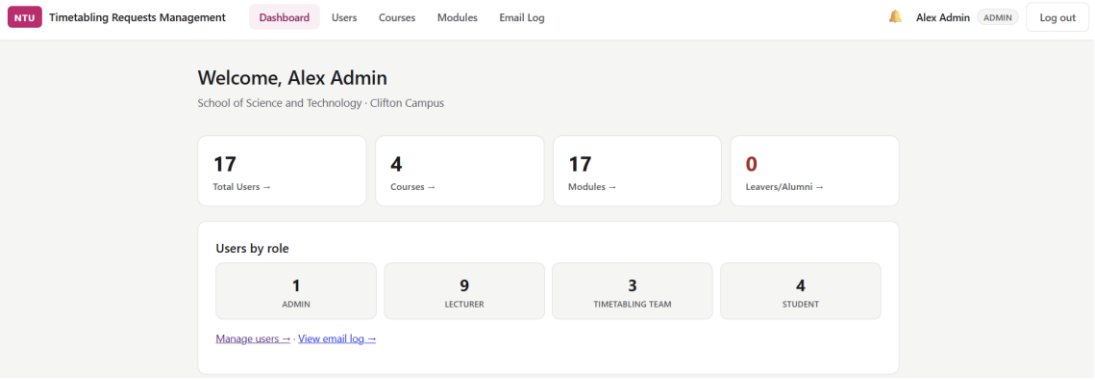

Figure 14. Screen 05 - Admin Dashboard

- **Manage Users** (create/edit, auto-emailed password, mark Leaver/Alumni)

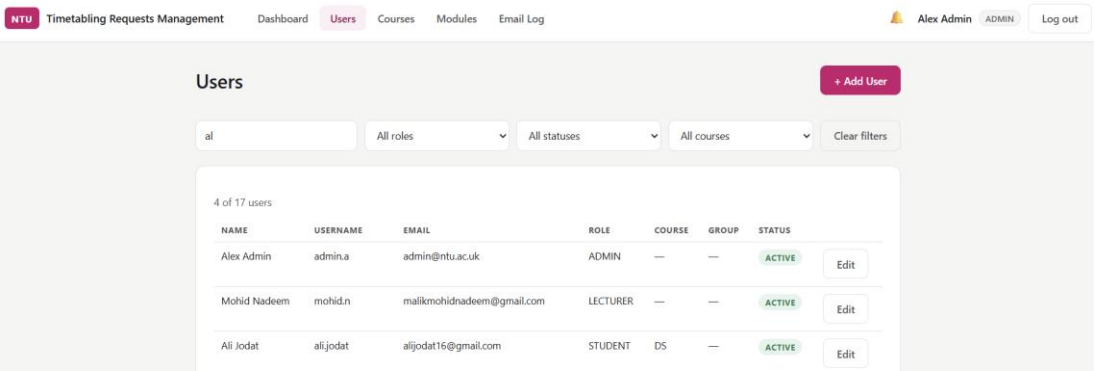

Figure 15. Screen 06 - Manage Users

- **Manage Courses** (CRUD, delete guards)

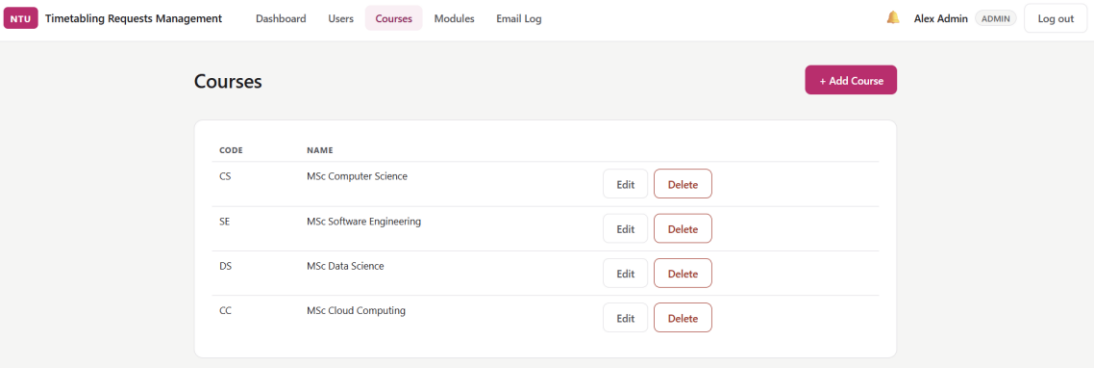

Figure 16. Screen 07 - Manage Courses

- **Manage Modules** (CRUD, course assignment, delete guards)

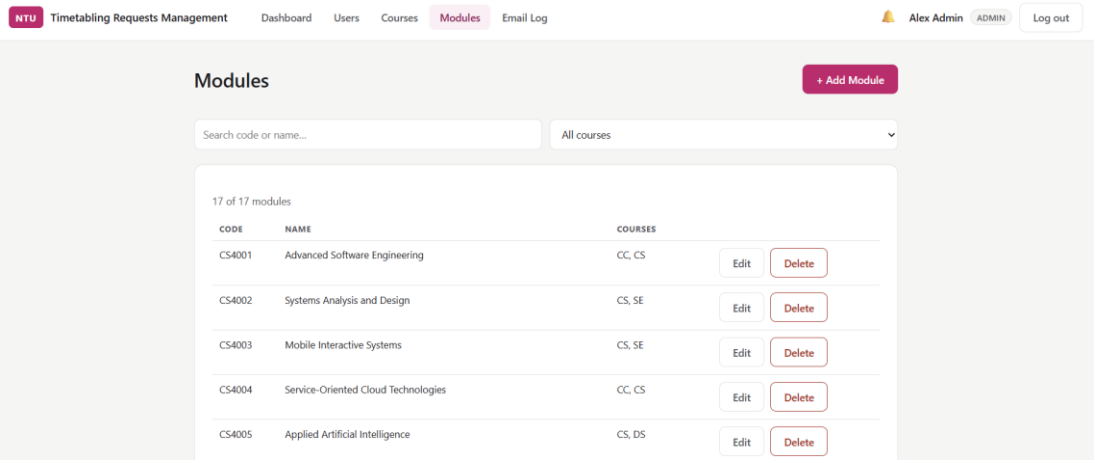

Figure 17. Screen 08 - Manage Modules

- **Email Log** (all outgoing email attempts, with filters)

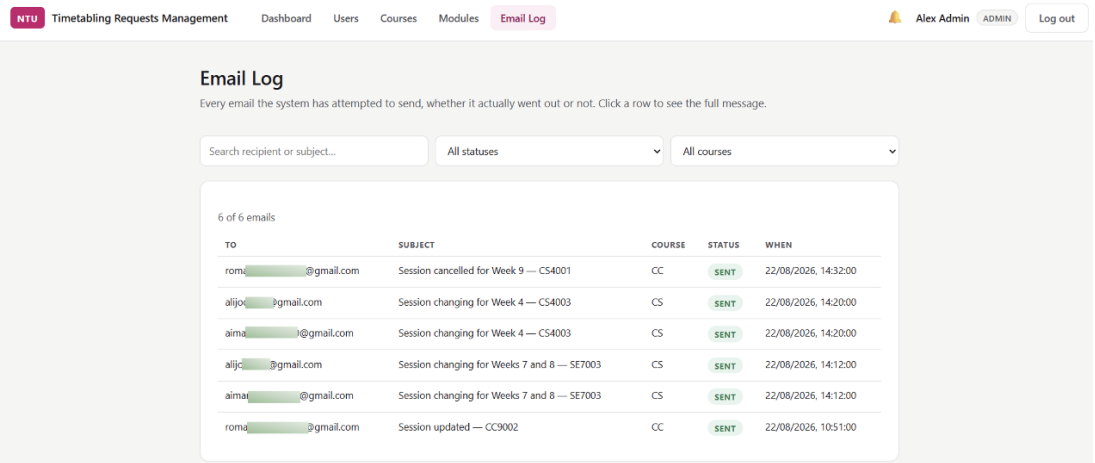

Figure 18. Screen 09 - Email Log

4.4.3. Lecturer Screens

- **Lecturer Dashboard**

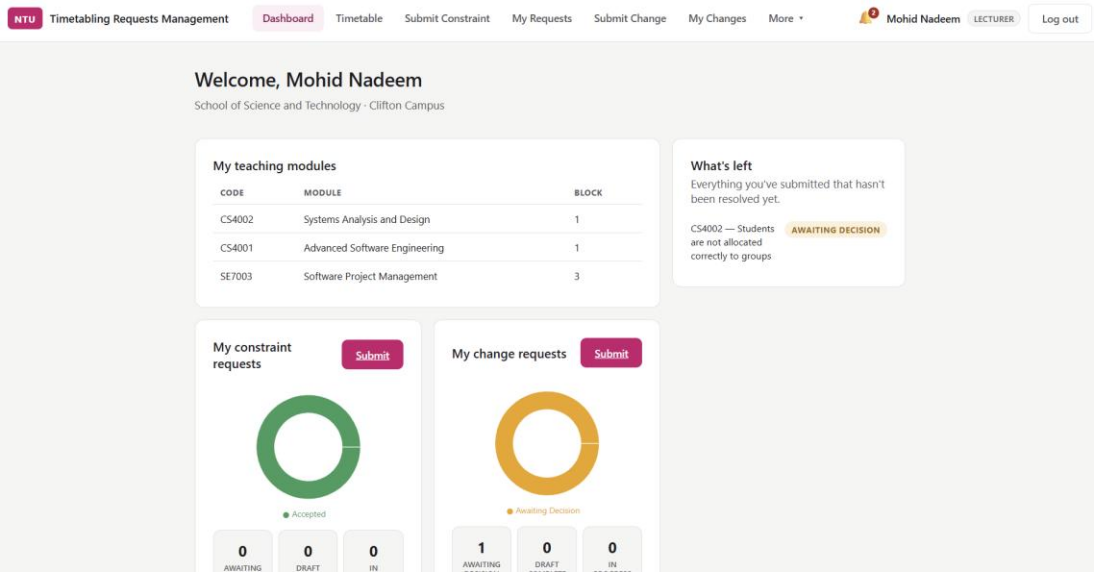

Figure 19. Screen 20 - Lecturer Dashboard

- **Submit Constraint** (Module-based / Personal) *(wireframe shown above — final screen to follow)*

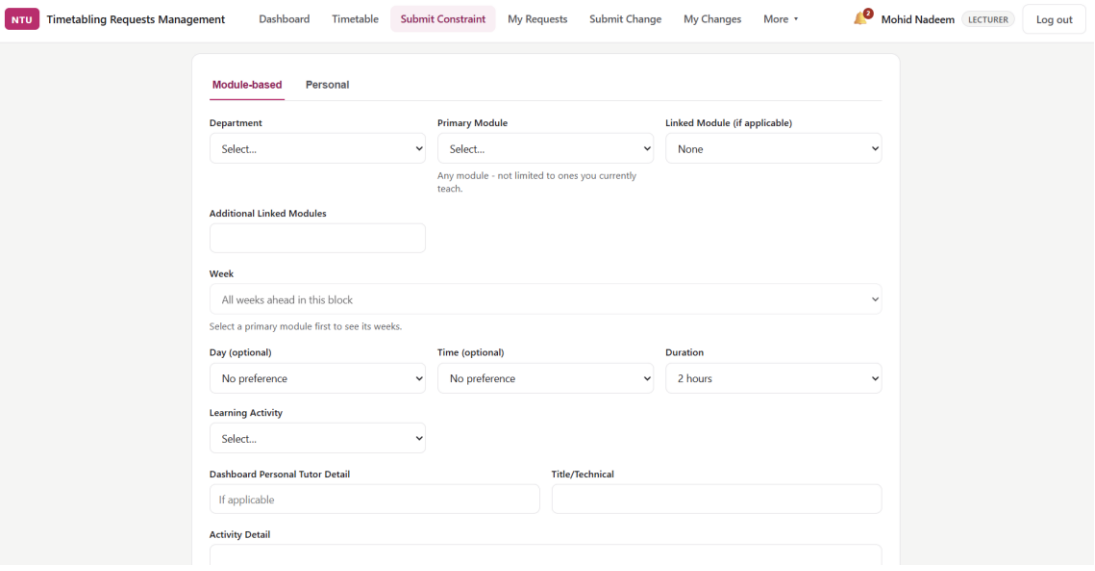

Figure 20. Screen 21 - Submit Constraint (Module-based)

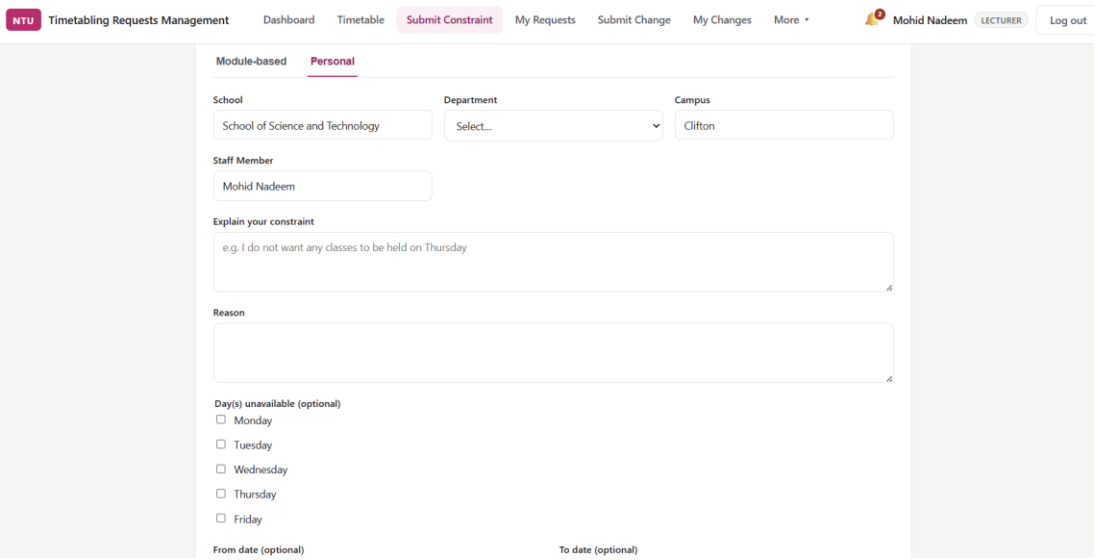

Figure 21. Screen 21 - Submit Constraint (Personal)

- **Submit Change Request** (category-first flow)

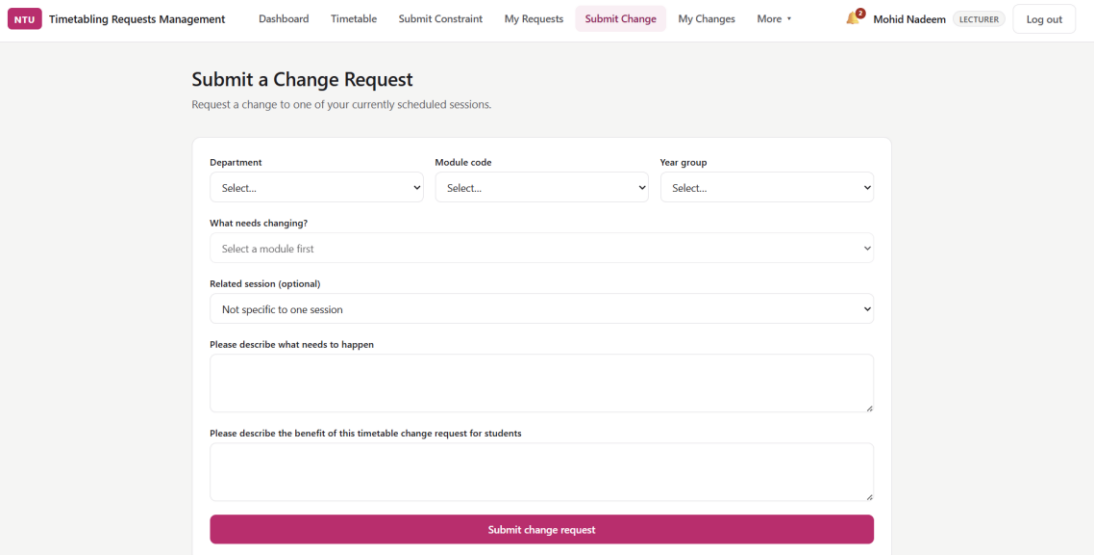

Figure 22. Screen 22 - Submit Change Requests

- **My Requests** (constraint + change request list)

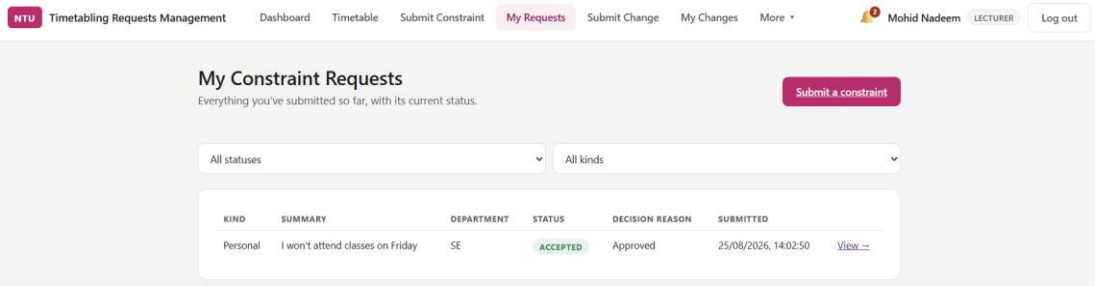

Figure 23. Screen 23 - My Constraint Requests

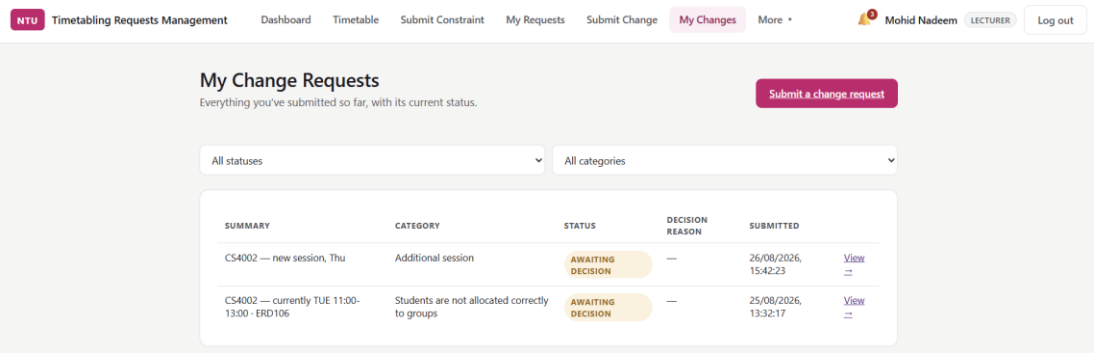

Figure 24. Screen 24 - My Change Requests

- **Request Detail** (status, decision reason, two-way communication thread)

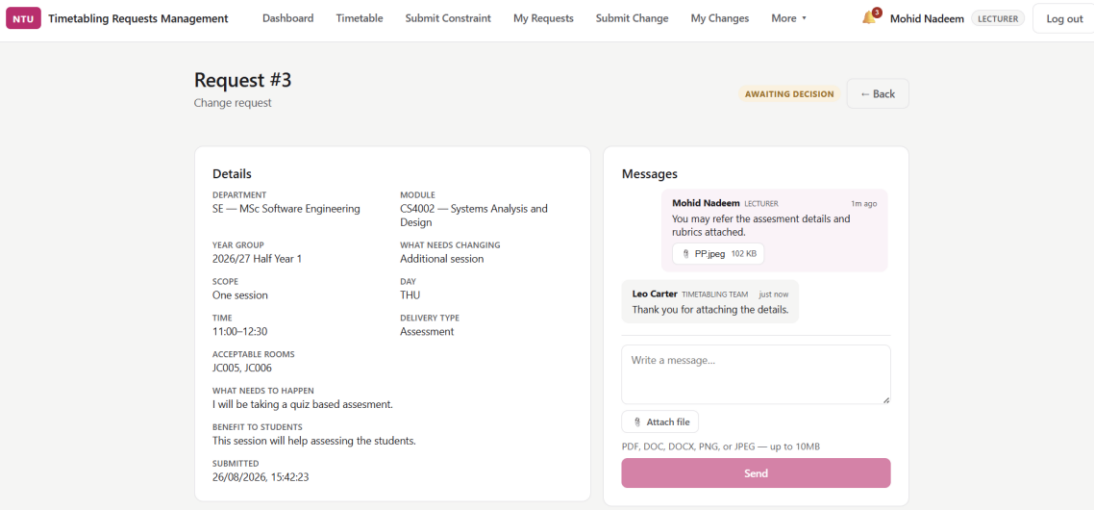

Figure 25. Screen 25 - Request Details and Chat Thread (Lecturer)

4.4.4. Timetabling Team Screens

- **Timetabling Team Dashboard**

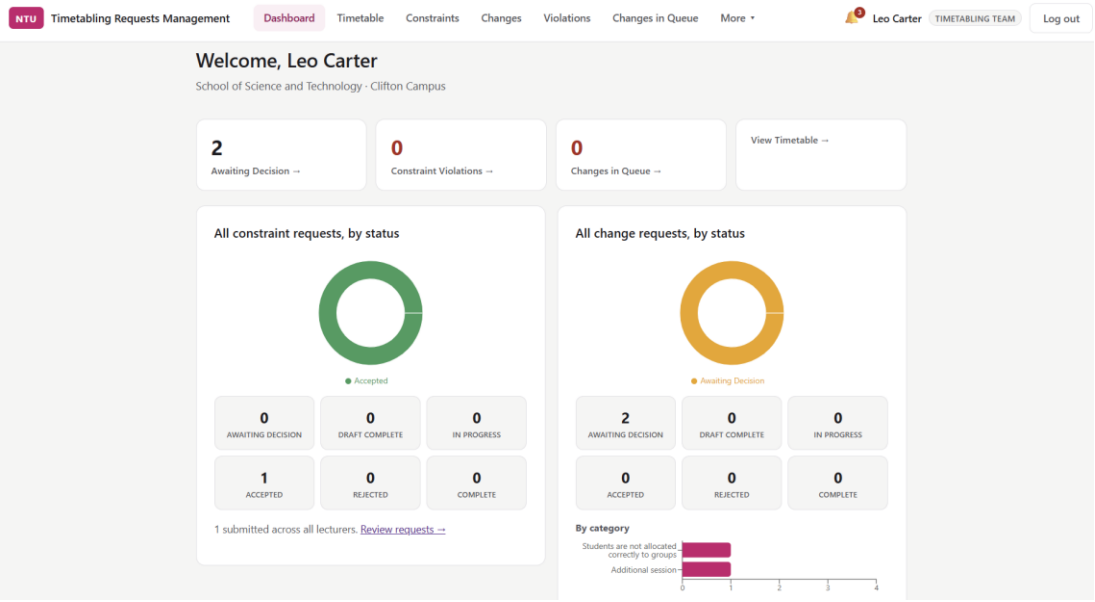

Figure 26. Screen 26 - Timetabling Team Dashboard

- **Constraints and Change Requests Queue**

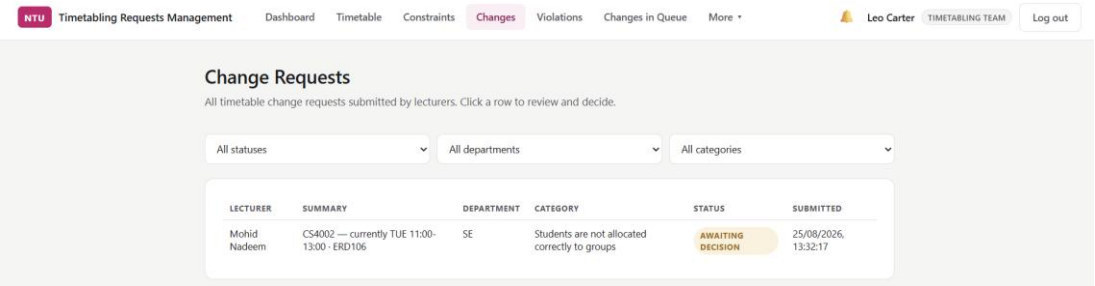

Figure 27. Screen 27 - Change Requests Queue

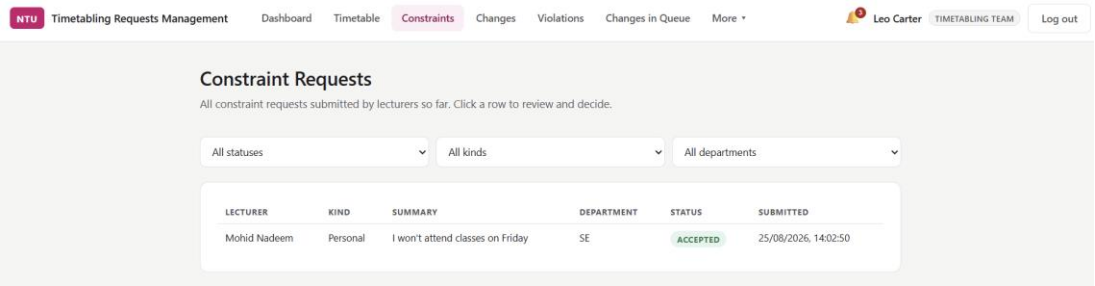

Figure 28. Screen 28 - Constraint Requests Queue

- **Violations** *(clash/conflict view against the live timetable — FR4)*

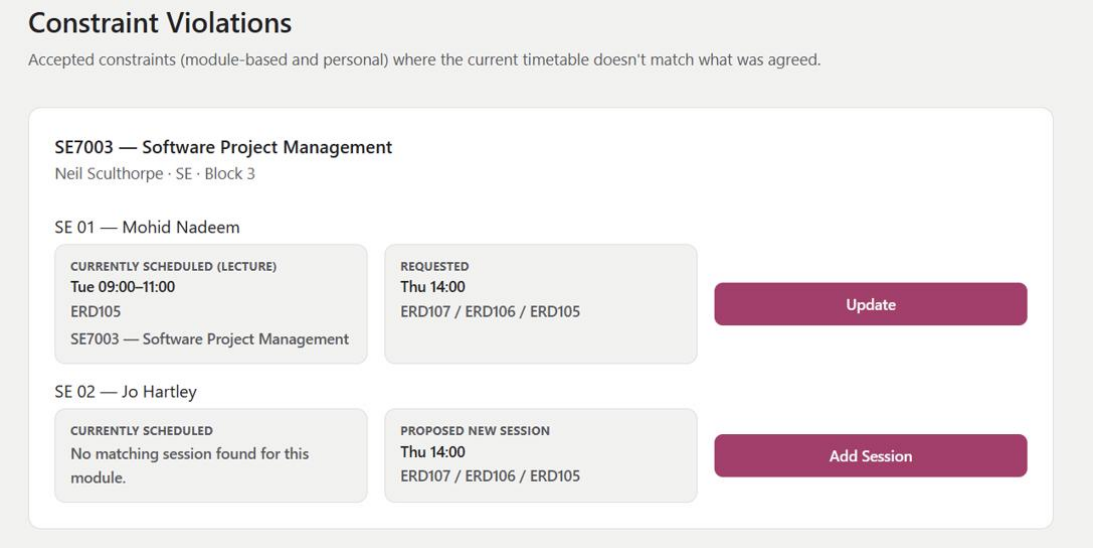

Figure 29. Screen 29 - Violations (based on Accepted Constraint Requests)

- **Changes in Queue**

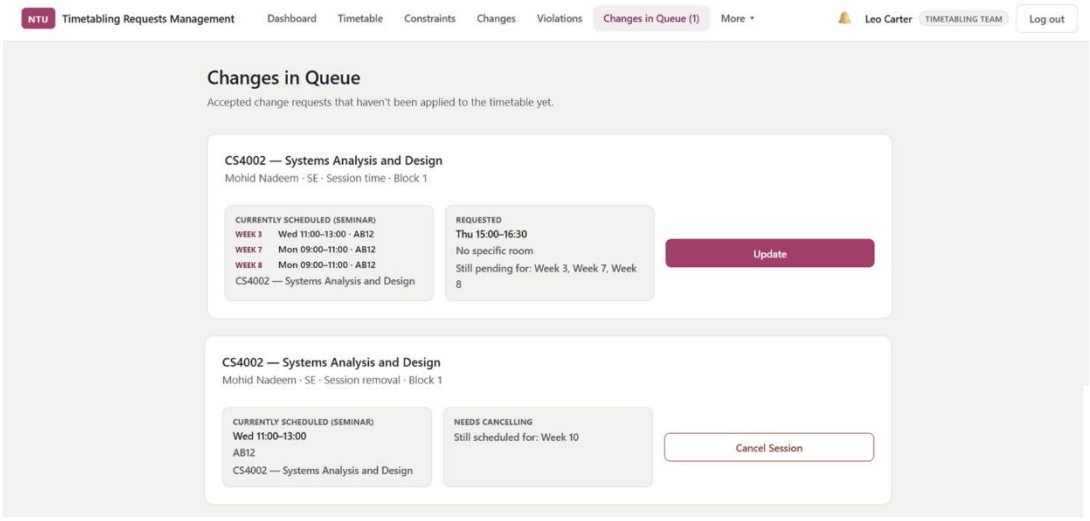

Figure 30. Screen 30 - Changes in Queue (based on Accepted Change Requests)

- **Request Detail** (review, action/approve/reject, decision reason, two-way communication)

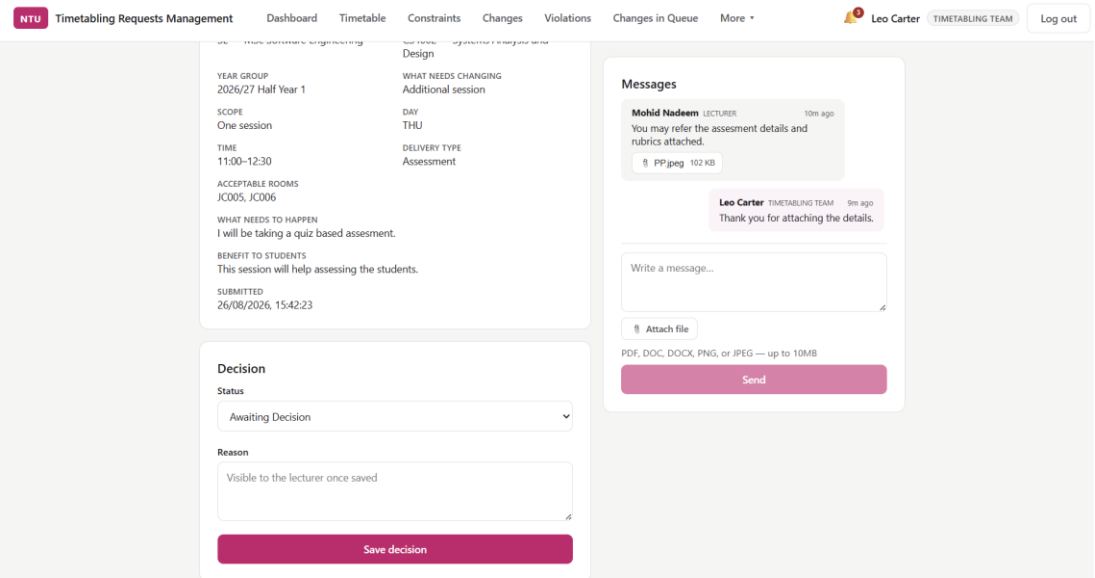

Figure 31. Screen 31 – Action on Request and Chat Thread (Timetabling Team)

- **Session Management** (update/add session without request, update academic year)

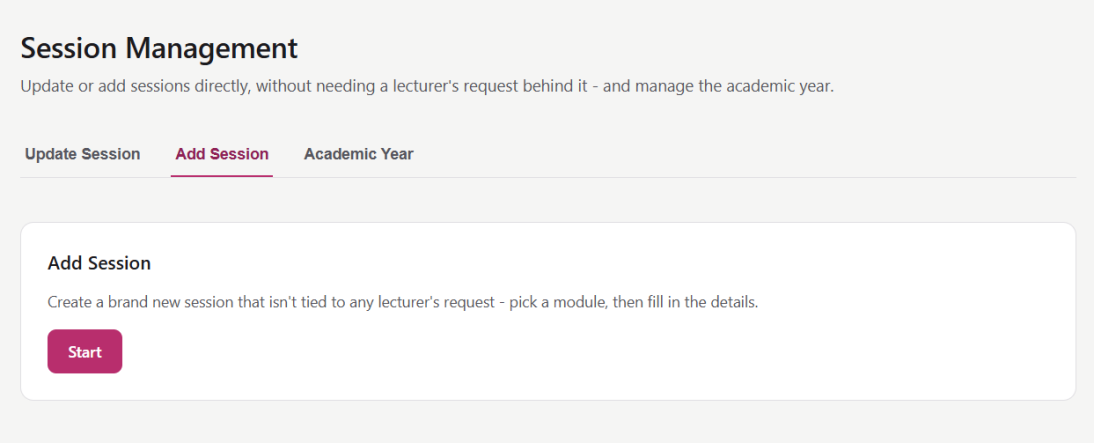

Figure 32. Screen 32 - Session Management

4.4.5. Student Screens

- **Timetable view, Notifications, Activity Log** (scoped to own course)

  (Refer to 4.4.1 – Shared Screens)

- **Update via Email**

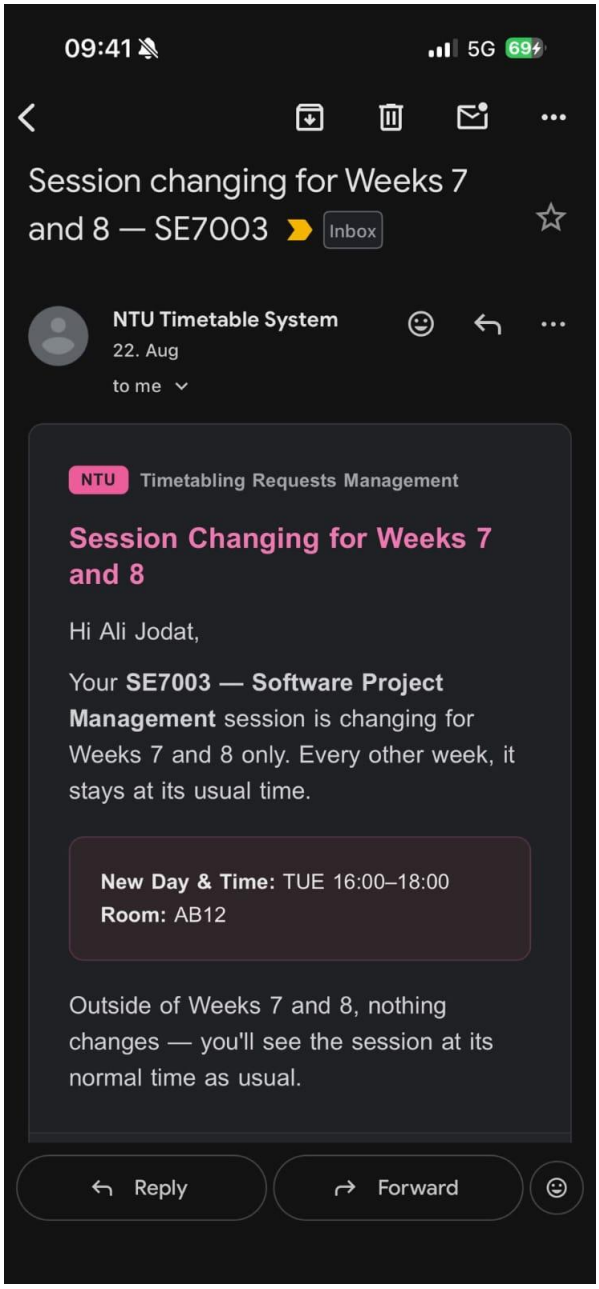

Figure 33. Email Thread – Screenshot provided by a student (permitted to use)

CHAPTER 5: EVALUATION

5.1. Evaluation Approach

Evaluation was carried out throughout development rather than as a single final activity, to match with the iterative feedback and refinement approach discussed in the Literature Review (Section 5.4), where Larman and Basili (2003) and Gould and Lewis (1985) both emphasise that usability improves when users are involved continuously rather than only at the end. Three separate evaluation activities were carried out, each suited to how that stakeholder group actually interacts with the system:

- The **lecturer** participant was involved incrementally, through two full evaluation sessions conducted mid-development, providing detailed qualitative feedback that directly shaped subsequent development work.

- The **Timetabling Team** completed a formal usability evaluation, combining a System Usability Scale (SUS) with three feature-specific satisfaction ratings, after a short demo of the developed prototype.

- **Students**, who majorly interact with the system through email notifications (Section 4.1.7), were surveyed specifically on the clarity and usefulness of the notification emails they received during testing.

5.2. Lecturer Evaluation (Incremental, Two Sessions)

Two evaluation sessions were held with the lecturer participant during development, each resulting in a set of concrete, actionable points rather than general impressions, in keeping with the semi-structured, conversational style used throughout requirement elicitation (Section 4.1.2).

The **first session** focused on the constraint and change request submission flows. Feedback here directly reshaped a core design decision: module-based constraints were originally tied to the individual lecturer submitting them, but the participant clarified that a constraint should belong to the module itself, irrespective of who currently teaches it, since staffing can change while the underlying scheduling constraint remains valid. This was implemented as a redesign rather than a minor fix. Further points from this session covered the need for more realistic mock timetable data (sessions varying by week rather than being fixed all year, and lab groups needing separate teacher assignment), which then was implemented through revising the mock database accordingly (Section 4.2.2).

The **second session**, held once the dashboard, timetable, and request-handling features were further along, produced a longer set of points, most of which were addressed directly:

Table 6. Feedback and Actions - Evaluation Meeting 02

Feedback point	Outcome
Dashboard shouldn't lead with statistics; lecturer wants to see "which problems haven't been fixed yet"	Redesigned
Timetable should default to showing the logged-in user's own sessions, not everyone's	Implemented
Intersecting filters (course and person at once)	Confirmed as a genuine improvement over NTU's existing system, kept as-is
Redundant "who's delivering this session" field	Removed
"Online" sessions should be handled more simply	Simplified to a single seeded pseudo-room
Clashes/merge session pickers showing irrelevant options	Filtered to genuinely relevant sessions only

5.3. Timetabling Team Evaluation (Short Demo + SUS)

Following a short demonstration of the completed prototype, two Timetabling Team members completed a usability evaluation combining SUS questionnaire with three feature-specific satisfaction ratings. It's worth noting upfront that the questionnaire used included seven of the standard ten SUS statements.

SUS-style items (1 = Strongly Disagree, 5 = Strongly Agree)

Table 7. SUS Evaluation Response Summary

Statement	Respondent 1	Respondent 2	Mean
I would like to use this system frequently	4	3	3.5
I found the system unnecessarily complex *(negative)*	2	2	2.0
I thought the system was easy to use	5	4	4.5
I would need technical support to use this system *(negative)*	3	1	2.0
I thought there was too much inconsistency *(negative)*	1	2	1.5
I found the functions well integrated	4	3	3.5
I think most people would learn this quickly	4	4	4.0

Taken together, the pattern is positive: both respondents rated ease of use highly, rated complexity and inconsistency low, and rated the need for technical support low.

The responses are consistent, only "would need technical support" shows a notable split (3 vs 1).

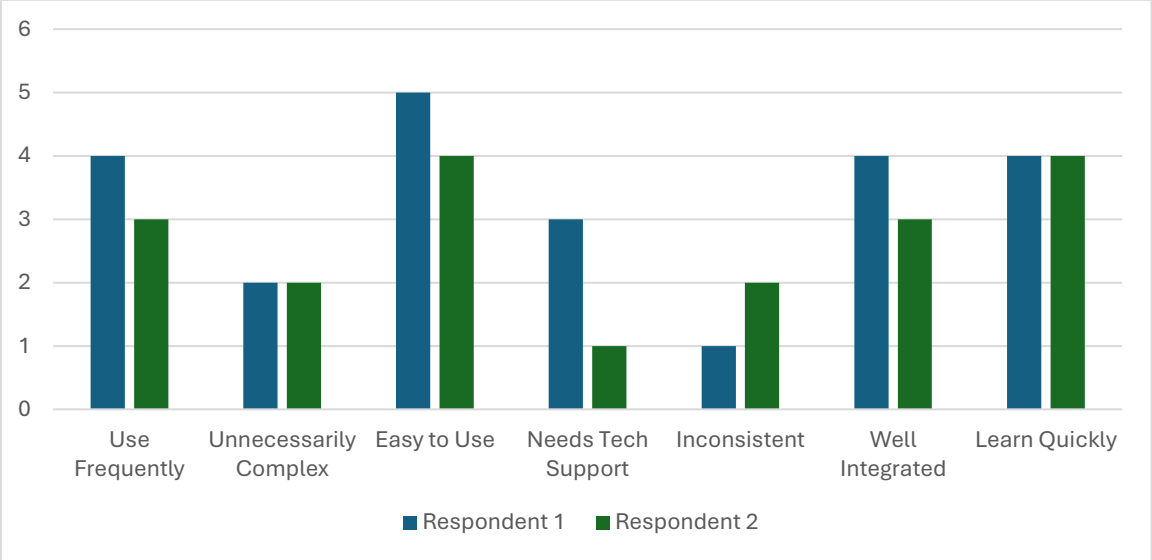

Figure 34. Chart - SUS Response Summary

Feature-specific satisfaction (out of 5)

Both respondents rated all three core feature areas highly, with Notifications & Two-way Communication and Change Requests rated marginally higher than Constraint Requests.

Table 8. Feature-Specific Response Summary

Feature	Respondent 1	Respondent 2	Mean
Constraint Requests	4	4	4.0
Change Requests	5	4	4.5
Notifications & Two-way Communication	5	4	4.5

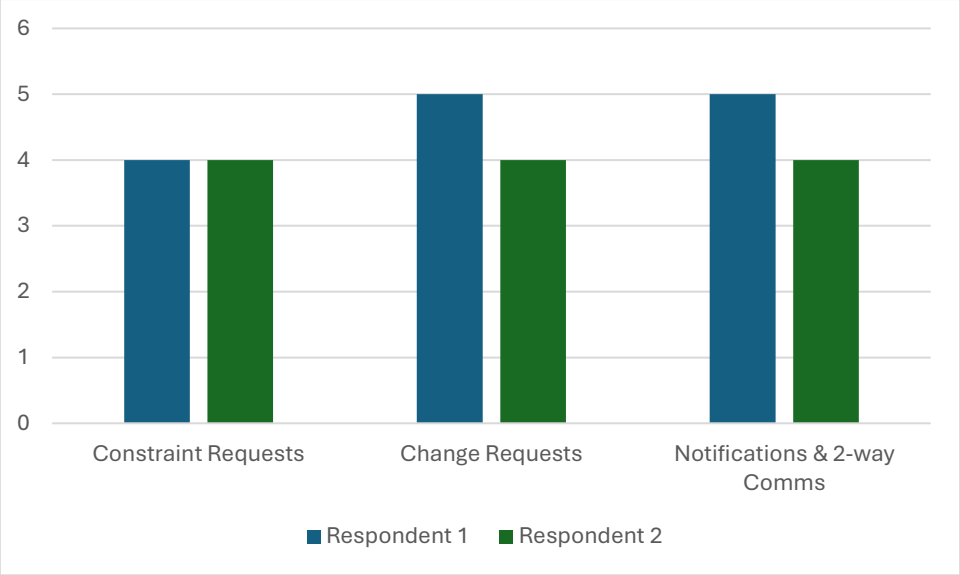

Figure 35. Chart - Feature-Specific Rating

5.4. Student Evaluation (Notification Email Usability)

Eight students completed the evaluation survey after receiving an automated timetable notification email during testing. All eight confirmed they had received the email, so no responses were excluded.

Table 9. Student Evaluation Response Summary

Measure	Result
Ease of understanding what happened to the session	7 x "Very easy", 1 x "Easy"
Clarity of information in the email	6 x "Very clear", 2 x "Clear"
Ease of identifying key details (day, time, room, tutor)	6 x "Very easy" 2 x "Easy"
Would this approach improve communication at NTU?	7 x "Probably yes" 1 x "Definitely yes"

No respondent identified missing information that should be added to the email, one noted that a specific date wasn't included but concluded it wasn't necessary, since the week number and day already made it clear. Free-text comments on the design were uniformly positive or neutral ("it's all good," "clear and well-designed," "very accessible"), with no respondent suggesting a change.

This result directly supports the original requirement elicitation finding (Section 4.1.4) that students wanted real-time, clearly understandable notifications without needing to check a calendar manually, and suggests the implemented email format (Section 4.3.2, Increment 4a) succeeded in meeting that expectation.

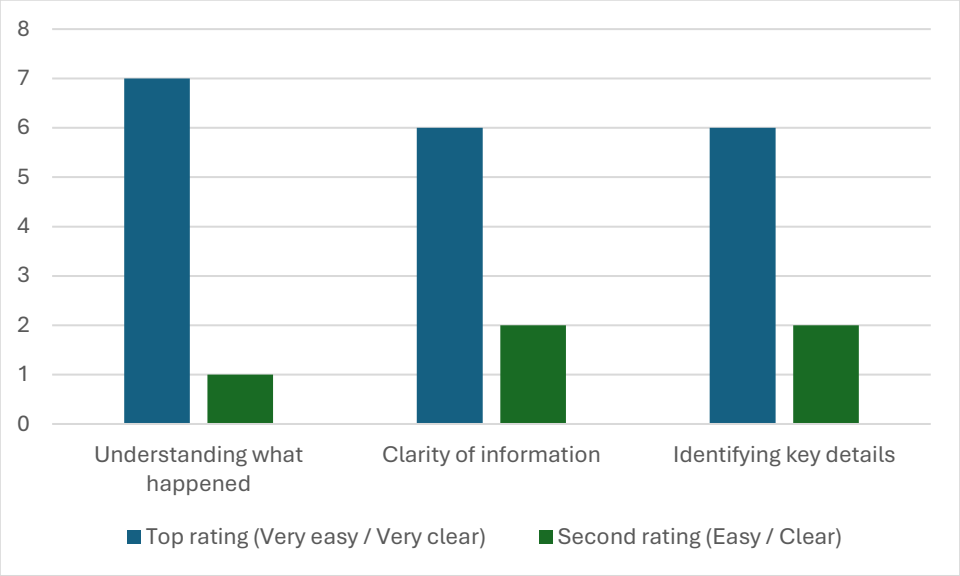

Figure 36. Student Evaluation Response on Email Clarity

5.5. Evaluation Limitations

- The Timetabling Team SUS evaluation used a shortened, seven-item version of the standard ten-item SUS questionnaire. Results are reported descriptively rather than as a single benchmark figure.

- Both the Timetabling Team evaluation (n=2) and the qualitative lecturer evaluation (n=1) reflect small sample sizes. As discussed in the Literature Review (Section 5.5), Virzi (1992) suggests even small samples can surface a meaningful proportion of usability issues, but these results should still be read as indicative rather than statistically representative of their wider stakeholder groups.

- The student survey (n=8) evaluated only the notification email in isolation, since that is the only part of the system students have the most interest in.

CHAPTER 6: CONCLUSIONS

6.1. Findings

The outcome of the project is a prototype that demonstrates a communication-focused timetabling system, built around structured requirement elicitation and iterative stakeholder feedback. It can improve how timetable-related communication is handled compared to the current fragmented process.

The evidence for this comes from the consistency between what stakeholders asked for and how they rated the finished prototype. Progress visibility and two-way communication, the most strongly requested features across the lecturer and Timetabling Team interviews (Section 4.1.5), were also the two features rated most highly in the Timetabling Team's evaluation (Section 5.3).

Similarly, students asked for clear, real-time notifications without needing to manually check a calendar (Section 4.1.4), and the notification email they were later tested on was rated positively by all eight respondents, with no requests for additional information or design changes (Section 5.4).

This alignment between stated requirements and evaluated outcomes suggests the hybrid elicitation approach adopted (Chapter 3, Literature Review Section 4.5) was effective at surfacing genuine and actionable needs rather than assumptions.

The lecturer's two incremental evaluation sessions also demonstrated the practical value of the iterative-incremental development approach justified in Chapter 3. The way the requirements emerged or became clearer through earlier feedback would likely have been missed or would have been identified at the very end if the waterfall process would have been followed.

6.2. Achievement of Aims and Objectives

Revisiting the SMART objectives set out in Chapter 1:

Table 10. Objectives and their Outcomes

Objective	Outcome
Conduct literature review and background research	Achieved (Chapter 2)
Gather and analyse stakeholder requirements	Achieved, across three stakeholder groups via interviews and surveys (Chapter 4)
Design a prototype architecture and user workflow	Achieved, including use case diagrams, ERD, and wireframes (Chapter 4)
Develop the prototype incrementally	Achieved, across six increments (0, 1, 2, 3, 4a, 4b)

Evaluate the prototype using usability techniques	Achieved, via lecturer evaluation sessions, Timetabling Team SUS/feature ratings, and student email survey (Chapter 5)
Analyse evaluation results and refine the prototype	Achieved, most visibly through the lecturer's two incremental evaluation sessions directly shaping development

The overall aim, to design, develop, and evaluate a communication-focused timetabling support prototype that improves stakeholder interaction, scheduling coordination, and change management, has been met within the scope defined in Chapter 3. The prototype does not replace or integrate with NTU's live timetabling system and was never intended to; rather the major intention was to make a system which can help improving the stakeholder involvement and evaluation throughout the timetable modification process.

6.3. Limitations

- **Small sample sizes** across all three evaluation activities (one lecturer, two Timetabling Team members, eight students) limit how confidently these findings generalise, as discussed throughout Chapters 4 and 5 with reference to Virzi (1992).
- **The mock timetable database**, a copy of the real timetable but with limited courses and modules, which means the prototype has not been tested against the full scale, complexity, or edge cases of a real institutional timetable.
- **The system deliberately does not write to or integrate with NTU's actual timetabling software.** This was a conscious scope decision, confirmed directly with the supervisor, The prototype remains a proof-of-concept for the communication workflow rather than a deployable replacement. The concept can be integrated into the real system accordingly.
- **No student-level enrolment data exists in the system.** Certain change request categories (e.g. Student Allocation and automatically calculating the effect of merging sessions) rely on knowing which students belong to which group, data that sits within NTU's actual student records system and was kept out of scope for this project.

6.4. Future Work

Several extensions were identified during development and evaluation but were scoped out, either due to time constraints or because they depend on data/integration this project deliberately didn't take on.

- **Integration with NTU's live timetabling and student record systems:** The most significant extension would be connecting the system to real timetable and enrolment data rather than the seeded mock database, enabling accurate, automatic calculation of the effect of a request (e.g. which students are actually affected by a session merge or reallocation), something the current system cannot do without that data.

- **Multi-teacher session support:** Sessions such as labs are sometimes split across multiple staff members, each responsible for a different group. This was explicitly raised during the lecturer's second evaluation session and marked out of scope for the prototype but would be a natural next increment.

- **AI-assisted request handling:**

    - **Conflict/clash resolution suggestions** rather than only flagging that a requested time/room isn't available (FR4), the system could suggest alternative slots or rooms based on existing availability, similar to how calendar tools suggest meeting times.

    - **Pattern-based prioritisation** surfacing which pending requests are most likely time-sensitive or high-impact (e.g. affecting the most students) to help the Timetabling Team sort their queue.

- **Institutional single sign-on (SSO):** Noted as a "final system" consideration rather than a prototype requirement from the outset (NFR4, Section 4.1.6), as SSO integration would be a sensible step for any real deployment.

- **Notification preferences:** Students and staff currently receive all relevant notifications by default. Allowing users to customise which types of change they're notified about can be a reasonable enhancement, though never explicitly requested by stakeholders during this project.

- **Revisiting clash visibility on the general Timetable view.** It was marked a low priority but can be a good improvement for visual representation of any clash (if exists) on the timetable view.

6.5. Conclusion

This project set out to investigate communication challenges within university timetabling processes and to design, develop, and evaluate a prototype addressing them, specifically for the Nottingham Trent University.

Through a hybrid requirement elicitation approach spanning three stakeholder groups, an iterative-incremental development process, and evaluation activities tailored to how each stakeholder actually interacts with the system, the project has produced a working prototype that stakeholders rated positively across usability and feature-specific measures.

The future work identified above, particularly the real data integration and AI-assisted request handling, offers a reasonable foundation for this prototype to inform further development of NTU's timetabling infrastructure going forward.

REFERENCES

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D., 2001. *Manifesto for Agile Software Development*. Available at: Agile Manifesto Official Website [Accessed 16 May 2026].

Beyer, H. and Holtzblatt, K., 1998. *Contextual design: Defining customer-centered systems*. San Francisco: Morgan Kaufmann Publishers.

Boehm, B.W., 1988. *A spiral model of software development and enhancement*. Computer, 21(5), pp.61–72.

British Psychological Society, 2021. *Code of Human Research Ethics*. 4th ed. Leicester: British Psychological Society.

Brooke, J., 1996. *SUS: A "quick and dirty" usability scale*. In: P.W. Jordan, B. Thomas, B.A. Weerdmeester and I.L. McClelland, eds. Usability Evaluation in Industry. London: Taylor & Francis, pp.189–194.

Burke, E.K., Jackson, K., Kingston, J.H. and Weare, R.F., 1997. *Automated university timetabling: The state of the art*. The Computer Journal, 40(9), pp.565–571.

Dieste, O. and Juristo, N., 2011. *Systematic review and aggregation of empirical studies on elicitation techniques*. IEEE Transactions on Software Engineering, 37(2), pp.283–304.

Gould, J.D. and Lewis, C., 1985. *Designing for usability: Key principles and what designers think*. Communications of the ACM, 28(3), pp.300–311.

Information Commissioner's Office (ICO), 2024. *Guide to the UK General Data Protection Regulation (UK GDPR)*. Available at: ICO Official Website [Accessed 24 May 2026].

Larman, C. and Basili, V.R., 2003. *Iterative and incremental developments: A brief history*. Computer, 36(6), pp.47–56.

Loucopoulos, P. and Karakostas, V., 1995. *System requirements engineering*. London: McGraw-Hill.

McCollum, B., 2007. *A perspective on bridging the gap between theory and practice in university timetabling*. In: E.K. Burke and H. Rudová, eds. Practice and Theory of Automated Timetabling VI. Berlin: Springer, pp.3–23.

Nielsen, J., 1994. *Usability engineering*. San Francisco: Morgan Kaufmann.

Nuseibeh, B. and Easterbrook, S., 2000. *Requirements engineering: A roadmap*. In: Proceedings of the Conference on the Future of Software Engineering. New York: ACM, pp.35–46.

Royce, W.W., 1970. *Managing the development of large software systems*. In: Proceedings of IEEE WESCON. Los Angeles, California, pp.1–9.

Sommerville, I. and Sawyer, P., 1997. *Requirements engineering: A good practice guide*. Chichester: John Wiley & Sons.

Virzi, R.A., 1992. *Refining the test phase of usability evaluation: How many subjects is enough?* Human Factors, 34(4), pp.457–468.

APPENDIX A

Given the length of the main report, supplementary materials referenced throughout (interview transcripts, survey data, database schema, source code, and user documentation) have not been reproduced in full within this document. Instead, they have been uploaded separately via Dropbox and GitHub, as detailed in the table below.

Item	Location	Notes
Participant Consent form **Participant Information Sheet** **List of Questions** **Risk Assessment Form**	NTU Dropbox + GitHub Repo 03	Section 4.1.1
Interview transcripts - Snippets		Anonymised; Sections 4.1.2–4.1.3
Survey raw data (Student, Timetabling Team)		CSV exports; Section 4.1.4
Evaluation raw data (SUS, feature ratings, student email survey)		CSV exports; Chapter 5
Full database schema (schema.sql)		Section 4.2.2
Source code — Backend (Java/Spring Boot)	NTU Dropbox + GitHub Repo 01	
Source code — Frontend (React)	NTU Dropbox + GitHub Repo 02	
User Documentation	GitHub Repo 03	Not included in Dropbox submission

GitHub Repositories

Repo 01: 16 Commits as of 25th August 2026
https://github.com/MohidNadeem/NTU_Timetable_System_Backend_Mohid

Repo 02: 16 Commits as of 25th August 2026
https://github.com/MohidNadeem/NTU_Timetable_System_Frontend_Mohid

Repo 03: https://github.com/MohidNadeem/NTU_Timetable_System_Supporting_Files